



SCHOOL OF ADVANCED TECHNOLOGY
SAT301 FINAL YEAR PROJECT

Research on Image Preprocessing Strategies in PCB
Defect Detection

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Engineering

Student Name	:	Kaiqian Xiong
Student ID	:	2255533
Supervisor	:	Prof. Dechang Xu
Assessor	:	Pengfei Song

Abstract

Printed Circuit Board (PCB) defect inspection is important for quality control in electronics manufacturing. As PCB layouts become denser, small defects such as mouse bite and short defects become harder to detect. Their edges are weak, their contrast is low, and they can easily mix with surrounding circuit patterns. Many studies on PCB defect detection focus on improving deep learning models. In this project, we pay more attention to the preprocessing stage, mainly the effects of YOLOv8 data augmentation and Contrast Limited Adaptive Histogram Equalization (CLAHE) on YOLOv8n detection performance.

Six experimental groups were designed using mouse bite and short defects. Without data augmentation, the model achieved an mAP50 of 0.797 on the independent test set. After using YOLOv8 default augmentation, the mAP50 increased to 0.907. However, when default CLAHE was applied only during inference, the result dropped to 0.723 without augmentation and 0.840 with augmentation. This means that CLAHE does not always improve detection if it is added only at the inference stage. A main part of this study was therefore to search for better CLAHE parameters for this PCB dataset. The grid search showed that the optimal CLAHE parameters were clipLimit = 1.0 and tileGridSize = (6,6). With these parameters, G5 reached an mAP50 of 0.895, which was much better than the default CLAHE group and close to the augmented model tested on original images.

Using the optimal parameters from G5, we further tested a training-inference consistency strategy. Original training images were mixed with optimal CLAHE-enhanced images, and the same CLAHE setting was used during inference. This group gave the best result, with an mAP50 of 0.910, and the AP for mouse bite defects increased to 0.877. Overall, the results show that preprocessing settings clearly affect PCB defect detection. The main contribution of this study is the experimental identification of task-specific optimal CLAHE parameters, while the consistency experiment further shows that fixed preprocessing should also be considered during model training.

Keywords: PCB defect detection; data augmentation; CLAHE; YOLOv8; domain shift

Acknowledgements

Writing this acknowledgement makes me realize that my undergraduate study is really coming to an end. Looking back, the beginning of university still feels very close to me. I still remember the day when I received my admission letter. At that time, I was excited, but also uncertain about what kind of life I was going to face. Now, after finishing this Final Year Project, I have a similar feeling again. It is not only excitement, but also relief, gratitude, and a little sadness.

First, I would like to thank my FYP supervisor, Professor Xu Dechang. Since I did not come from a computer science background, this project was not easy for me at the beginning. Many concepts, methods, and technical details were unfamiliar, and I often felt that I did not know where to start. Professor Xu gave me patient guidance throughout the whole process, from understanding the basic ideas of PCB defect detection and building the project framework, to preparing group meeting reports and revising the thesis. His advice helped me gradually turn a vague topic into a complete project. More importantly, when I felt anxious about working in an unfamiliar field, his encouragement gave me confidence to continue.

I would also like to thank my parents. Studying in a different environment has brought many challenges, including language pressure, academic adjustment, and uncertainty about the future. During these years, their support has always been the most stable part of my life. When I first entered university, I was not fully prepared for the changes in study and daily life. My parents gave me not only financial support, but also emotional support when I needed it most. Their trust and patience helped me get through many difficult moments and allowed me to keep moving forward.

Finally, I want to thank myself—for not leaving that desolate version of me behind in that summer. “A smooth road leaves no clear footprints.” Carrying so many expectations, I

climbed out of the pit and have made it to where I am today. Looking back over the regrets scattered across these four years, the road no longer feels so treacherous.

The greatest regret in writing this acknowledgement is that my English is not proficient enough to quote and translate the profound verses of classical Chinese poetry to convey my feelings. Therefore, at the end of this acknowledgement, and at the end of my university life, let me conclude with the simplest words I have:

I am a storm bird, and I am not afraid of storms.

Contents

Abstract	ii
Acknowledgements	iv
Contents	vi
List of Tables	vii
List of Figures	viii
List of Acronyms	ix
Chapter 1 Introduction	1
1.1 Motivation, Aims and Objective	1
1.2 Literature Review	4
1.2.1 State of the Art in PCB Defect Detection	5
1.2.2 Image Preprocessing and CLAHE Techniques	6
1.2.3 Domain Shift and Training-Test Consistency	7
Chapter 2 Methodology and Results	9
2.1 Methodology	9
2.1.1 Dataset Description	9
2.1.2 Image Preprocessing Techniques	10
2.1.3 Experimental Group Design	13
2.1.4 Model Configuration	18
2.1.5 Evaluation Metrics	19
2.2 Results	21
2.2.1 Overall Performance Comparison	21
2.2.2 Baseline Effect of Training Augmentation (G1 vs G2)	25
2.2.3 Domain Shift Effect of CLAHE (G3 vs G1, G4 vs G2)	27
2.2.4 Mitigation Effect of Parameter Optimization (G5 vs G2)	31
2.2.5 Training-Inference Consistency Strategy (G6 vs G5, G6 vs G2)	38
2.2.6 Visual Case Analysis	41
2.2.7 Summary	47
Chapter 3 Conclusion and Future Work	50
3.1 Conclusion	50
3.1.1 Main Experimental Findings	50
3.1.2 Effect of Training-Inference Preprocessing Consistency	51
3.1.3 Practical Implications for PCB Defect Detection	52
3.2 Future Work	53
3.2.1 Dataset Expansion and Augmentation Strategy Exploration	54
3.2.2 Model Improvement and Real-Time Deployment	55
3.2.3 Generalization to Other Domains and Industrial Vision Tasks	56
References	57
Appendix A. Core Code	63
Appendix B. Supplementary Experimental Results	69
Appendix C. Environment and Configuration	71

List of Tables

Table	Title	Page
Table 2.1	Distribution of defect instances across the training, validation, and test sets	11
Table 2.2	Overview of experimental group configurations	15
Table 2.3	Overall performance comparison on the independent test set	22
Table 2.4	Performance comparison between G1 and G2	26
Table 2.5	Performance changes caused by default CLAHE during inference	28
Table 2.6	Performance comparison of G5 with G2 and G4	33
Table 2.7	Performance comparison of G6 with G5 and G2	39
Table 2.8	Confidence scores of G2, G5, and G6 for short defects in 04_short_15	41
Table 2.9	Confidence scores of G2, G5, and G6 for mouse bite defects in 05_mouse_bite_07	42
Table 2.10	Confidence scores of G2 and G6 for common short defects in detection_comparison_short	44
Table 2.11	Confidence scores of G2 and G6 for common mouse bite defects in detection_comparison_mb	45
Table A.1	The control methods for each experimental group	62
Table B.1	Full Grid Search Results	69
Table C.1	Statement of Software Environment	70
Table C.2	Statement of Hardware Specification	70
Table C.3	Statement of Training Hyperparameters	71

List of Figures

Figure	Title	Page
Figure 2.1	Bar chart comparing mAP50 among experimental groups	23
Figure 2.2	Grouped bar chart comparing AP for mouse bite and short defects	24
Figure 2.3	Bar chart comparing mAP50 between G1 and G2	26
Figure 2.4	Bar chart comparing AP for mouse bite defects and short defects between G1 and G2	27
Figure 2.5	Bar chart comparing mAP50 among G1, G3, G2, and G4	29
Figure 2.6	Grouped bar chart comparing AP for mouse bite defects and short defects among G1, G3, G2, and G4	30
Figure 2.7	Heatmap of CLAHE parameter grid search (based on G2 model mAP50 on the validation set)	34
Figure 2.8	Detection results of G2, G5, and G6 for short defects in 04_short_15	42
Figure 2.9	Detection results of G2, G5, and G6 for mouse bite defects in 05_mouse_bite_07	43
Figure 2.10	Detection results of G2 and G6 for short defects in detection_comparison_short	44
Figure 2.11	Detection results of G2 and G6 for mouse bite defects in detection_comparison_mb	46

List of Acronyms

Term	Initial components of the term
AOI	Automated Optical Inspection
AP	Average Precision
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CPU	Central Processing Unit
FPS	Frames Per Second
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	mean Average Precision
PCB	Printed Circuit Board
SGD	Stochastic Gradient Descent
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Motivation, Aims and Objective

Printed circuit boards (PCBs) are used in many electronic products, so their manufacturing quality is closely related to the performance and reliability of electronic devices^[1]. In recent PCB production, circuit layouts have become denser, and the distance between circuit lines has become smaller^[2]. Because of this, some defects are also small in the captured images and are not easy to separate from the background. Common PCB defects include mouse bites, shorts, opens, pinholes, spurs, and spurious copper^[3]. This project focuses on mouse bite and short defects. A mouse bite defect usually appears as a small notch on the edge of a circuit line, while a short defect appears as an unwanted connection between nearby conductors. These two types were selected because they are common PCB defect categories and are challenging under practical inspection conditions, where defect data may vary across different products, batches, and environments^[4].

Early PCB defect inspection mainly used manual checking or image processing methods based on hand-crafted features^[5]. These methods can still work in simple cases, but they are easily affected by human judgment, lighting changes, and image acquisition quality^[6]. For this reason, deep learning has become an important direction in PCB defect detection. Convolutional neural network (CNN)-based models can learn image features from training data and have been used for PCB inspection tasks^[7]. Among these models, the You Only Look Once (YOLO) series is often chosen because it gives a reasonable balance between speed and accuracy^[8]. Many previous studies have tried to improve YOLO-based PCB detection by changing network structures, adding attention modules, or improving feature fusion^[9].

This project takes a different angle. Instead of mainly modifying the model architecture, it studies whether preprocessing choices change the detection result. This question is worth testing because small PCB defects may not be clear in the original image, even when a deep learning model is used. Existing studies have discussed image preprocessing, but the separate effects of training-stage data augmentation and inference-stage contrast enhancement are still not very clear for this task^[1]. In general, preprocessing is used to improve image quality, make useful defect details easier to see, or reduce the influence of background noise^[10]. For mouse bite and short defects, this step may directly affect whether the model can extract useful local features. If the image itself does not show the defect clearly, the detector may still give weak or unstable predictions^[11].

Contrast Limited Adaptive Histogram Equalization (CLAHE) is one possible method for improving image contrast^[12]. It enhances local image regions instead of changing the whole image in the same way. It also limits the enhancement strength, which helps reduce the risk of making noise too strong^[13]. Besides contrast enhancement methods, attention-based mechanisms have also been explored to improve PCB defect classification performance^[14]. However, its effect is not fixed. In this project, the two main parameters are clipLimit and tileGridSize. Different values can make the enhanced PCB images look quite different. Some settings may make defect edges clearer, while other settings may strengthen background texture too much. Therefore, the CLAHE parameter setting itself becomes an important part of the experiment.

Data augmentation is another factor considered in this project. It changes the training images in different ways, such as rotation, scaling, flipping, and color adjustment, so that the model can learn from more image variations^[15]. This is useful for this project because the dataset is not large and the selected defects are small. However, industrial PCBA defect inspection still faces challenges such as complex backgrounds, multiscale defects, low-contrast regions, and limited training samples^[16]. Therefore, the effect of augmentation needs to be checked under this specific PCB dataset. Since mouse bite and short defects have weak visual features, the effect of augmentation needs to be checked under this

specific dataset. Previous studies also show that task-related augmentation settings can help improve small-object detection^[17].

Another problem is the difference between the images used for training and the images used during inference. If the model is trained on original PCB images but tested on CLAHE-processed images, the image style during inference is changed. This situation is related to domain shift, where the test data distribution is different from the training data distribution and the model performance may decrease^[18]. In this project, this problem is directly connected to CLAHE preprocessing. If CLAHE is only added during inference, the model has not learned from images with that enhanced style during training. This can be regarded as a specific case of domain shift, where the test data distribution differs from the training data distribution and model performance may decrease^[18]. This issue is also related to the broader generalization challenge in PCB defect detection under varying inspection conditions^[19].

Considering the issues discussed above, this project was designed around five main objectives.

1. To build a basic reference group.

The first step is to train and test YOLOv8n under the simplest setting, without using data augmentation or CLAHE. This experiment, marked as G1, provides the basic result for detecting mouse bite and short defects, and serves as the reference point for later comparisons.

2. To check whether data augmentation improves the model.

In the second experiment, YOLOv8 default data augmentation is enabled during training. This group, marked as G2, is compared with G1 so that the effect of data augmentation can be observed more directly.

3. To observe what happens when CLAHE is only used during inference.

In the next two experiments, CLAHE with default parameters is applied only at the inference stage. The parameters are set as clipLimit = 2.0 and tileGridSize = (8,8). G3 is based on the model trained without augmentation, while G4 is based on the model trained with augmentation. These two groups are used to examine whether a change in image style during inference leads to a mismatch with the training images.

4. To find a more suitable CLAHE setting for this dataset.

Since the default CLAHE setting may not be suitable for small PCB defects, a grid search is carried out in G5. Different combinations of clipLimit and tileGridSize are tested using the G2 model. The purpose is to select a better CLAHE parameter combination and to see whether this adjustment can reduce the loss caused by using CLAHE only during inference.

5. To test the consistency between training and inference preprocessing.

The final experiment, G6, uses a mixed training set. Original images are combined with images processed by the selected CLAHE parameters. During inference, the same CLAHE setting is also used. This design allows the model to see the target preprocessing style during training, and is used to examine whether such consistency can improve detection performance.

Overall, the six groups are arranged in a step-by-step manner. G1 and G2 compare the effect of data augmentation, G3 and G4 test the influence of inference-stage CLAHE, G5 examines parameter optimization, and G6 further tests whether matching the preprocessing used in training and inference is useful. Through these comparisons, this study focuses on how preprocessing choices influence YOLOv8n-based PCB defect detection, rather than only discussing changes in model architecture. The results are expected to give a practical reference for selecting preprocessing methods in PCB inspection tasks and other similar industrial image detection problems.

1.2 Literature Review

1.2.1 State of the Art in PCB Defect Detection

Printed circuit boards (PCBs) are used in most electronic products, so defects on a board may affect product quality and reliability. With the miniaturization of electronic devices, PCB layouts are becoming more compact, and some defects occupy only a very small area in the image. This makes inspection more difficult, especially when the defect is close to dense circuit patterns or has weak visual features. Earlier PCB inspection mainly relied on manual visual checking or image processing methods based on hand-crafted features. These methods can still be used in some simple cases, but they are usually slow and may be affected by operator experience, image quality, and lighting changes^[5].

Deep learning has become a common method for PCB defect detection in recent years. Compared with traditional hand-crafted feature methods, convolutional neural networks (CNNs) can learn useful image features from training data, which makes them suitable for object detection tasks. Among different detectors, the You Only Look Once (YOLO) series is often used in industrial inspection because it can process images quickly while maintaining relatively good detection accuracy^[20]. Nasri et al. compared YOLOv8 and YOLOv9 in PCB defect detection. Their results showed that YOLOv9 achieved a mean average precision (mAP) of 98.4% at an IoU threshold of 0.5 on a public bare-board PCB dataset^[20].

For PCB micro-defect detection, many studies have further improved the YOLO structure to better capture small and weak defect features. Song and Song proposed SNF-YOLOv8, namely Scale-aware Neural Fusion YOLOv8, by adding a lightweight multi-scale feature fusion strategy and an attention-based scale sequence fusion mechanism to YOLOv8n^[21]. On a PCB defect dataset containing 2,772 images, this model reached 90.0% mAP50, which was 3.1% higher than the original YOLOv8n. At the same time, the number of parameters was reduced by 17% to 2.49M, and the inference speed reached 250 FPS^[21]. For micron-scale mouse-bite defects, the detection accuracy increased by 7.2%, suggesting

that the improved feature fusion design was helpful for reducing false detections in complex backgrounds^[21].

Dataset construction is also an important basis for PCB defect detection research. Ancha et al. built and released a hybrid PCB defect dataset covering six common defect types, including missing hole, mouse bite, open circuit, short, spur, and spurious copper^[22]. In their work, all images were resized to 640×640 pixels. Data augmentation was also applied so that the dataset could contain more image variations and better reflect possible inspection conditions in practice^[22]. The authors further tested YOLO models on this dataset, so the study can be used as a reference for later PCB defect detection experiments^[22].

1.2.2 Image Preprocessing and CLAHE Techniques

Image preprocessing is usually carried out before images are used for model training or testing. In this step, the original image is adjusted so that the input given to the detection model is easier to use. For example, preprocessing may improve image brightness, reduce noise, or make object and defect regions clearer^[23]. In industrial inspection, this step is often needed because captured images may be affected by uneven lighting, low brightness, equipment condition, or other acquisition problems. These factors can make defects less visible and may also introduce extra noise into the image^[23].

Contrast Limited Adaptive Histogram Equalization (CLAHE) is a common method for contrast enhancement. Kanellakis et al. used CLAHE in a low-light 3D reconstruction task to improve the visibility of captured images^[23]. Although their application was not PCB defect detection, the same enhancement idea is still relevant here: when the original image has low contrast, local contrast enhancement may help make weak visual details easier to observe. Different from ordinary histogram equalization, CLAHE does not enhance the whole image in one global way. It divides the image into small local regions and applies

histogram equalization inside these regions. At the same time, a contrast limit is used to avoid making noise too strong. In this way, CLAHE can improve local details while reducing the risk of over-enhancement^[23].

For PCB defect detection, this balance is important. Mouse bite and short defects are usually small, and their visual features may be close to the surrounding circuit patterns. If enhancement is too weak, the defect area may still be unclear. If enhancement is too strong, background texture may also be strengthened and may disturb the detection model. Therefore, the effect of CLAHE depends not only on whether it is used, but also on how its parameters are set.

Ammar et al. combined CLAHE with fuzzy enhancement for industrial videoscope images^[10]. Their results showed that the combined method improved several image quality indicators and helped preserve image details^[10]. This study supports the use of CLAHE in industrial image enhancement. It also suggests that the final enhancement effect may depend on the specific setting and on whether CLAHE is used alone or together with other methods^[10].

1.2.3 Domain Shift and Training-Test Consistency

In practical model deployment, the test images may not look exactly the same as the training images. When this happens, the model may perform worse than expected because it has learned features from one image distribution but is evaluated on another. This problem is usually discussed as domain shift^[24]. In industrial inspection, such differences can come from changes in lighting, camera condition, image acquisition settings, or surface texture. These changes may make the test images different from the images used during training^[24].

Some studies try to reduce this problem through test-time adaptation, where the model is adjusted during inference to better fit the target data^[24]. Tang et al. proposed a method based on visual conditioning tokens. In their approach, domain-specific information from target samples is captured during test-time adaptation, while domain-related information in the input feature representation is gradually reduced^[24]. Their results showed that this method improved the adaptation ability of source models when they were tested across different domains^[24].

Tang et al. proposed another method, called Dual-Path Adversarial Lifting. In this method, domain-shift tokens are introduced into different layers of a transformer network. The method uses token prediction and token update processes to reduce the influence of domain shift during online test-time adaptation^[25]. This method uses a dual-path structure that combines token prediction and token update processes to adjust the model under domain shift conditions^[25]. Their study also discussed the role of adversarial learning between the prediction network and the update network, and suggested that this design is useful for improving domain adaptation performance^[25].

However, many PCB defect detection studies still focus more on improving the detection model or network structure, such as adding new detection heads, feature extraction modules, or loss functions^[26]. In this project, I look at a simpler but practical problem: whether the preprocessing used during inference should also be considered during training. For PCB defect detection, this problem appears when CLAHE is used only during inference. If the model is trained with original PCB images but tested with CLAHE-enhanced images, the image style changes at the testing stage. Therefore, this study checks whether this difference affects detection results.

Chapter 2

Methodology and Results

2.1 Methodology

2.1.1 Dataset Description

The image data in this project came from the PKU-Market-PCB public dataset^[27]. This dataset was released by Huang and Wei and includes 1,386 PCB images. It contains six defect categories: missing hole, mouse bite, open circuit, short, spur, and spurious copper^[27]. For this project, I did not use all six categories. The experiments only focused on mouse bite and short defects. These two categories were chosen because they are relatively small in the images and their edges are not always clear. In some samples, the defect area is also close to complex circuit patterns, which makes the detection task more difficult^[28].

After selecting the two target categories, the original images were filtered and cropped according to the needs of this project. Finally, 231 images were kept for the experiments. All images were resized to 640×640 pixels, which is the input size used for YOLOv8n in this work. The dataset was then divided into three parts: training set, validation set, and test set. The training set contained 147 images, about 63.6% of all images. The validation set contained 37 images, about 16.0%, and the test set contained 47 images, about 20.4%. The test set was kept for the final evaluation only. It was not used during model training or CLAHE parameter selection, so that the final test results could better show how the trained model worked on unseen images.

The labels were stored in YOLO format. Each image had a corresponding text file, and each line in the file recorded one defect object with its class label and bounding box

coordinates. In this project, class 0 was used for mouse bite, and class 1 was used for short. Table 2.1 gives the number of defect instances in each subset. In the training set, there were 339 mouse bite instances and 288 short instances, giving 627 annotated defects in total. The validation set included 70 mouse bite instances and 87 short instances, with 157 defects. The test set included 83 mouse bite instances and 116 short instances, with 199 defects. In total, the final dataset used in this project contained 492 mouse bite instances and 491 short instances, giving 983 annotated defects across 231 images.

Subset	Images	Mouse Bite Instances	Short Instances	Total Instances
Training	147	339	288	627
Validation	37	70	87	157
Test	47	83	116	199
Total	231	492	491	983

Table 2.1 Distribution of defect instances across the training, validation, and test sets

2.1.2 Image Preprocessing Techniques

Two preprocessing-related settings were considered in this project: data augmentation during training and Contrast Limited Adaptive Histogram Equalization (CLAHE) for image enhancement. These two settings were tested separately and then combined in later experimental groups.

1. Data Augmentation

Data augmentation was used only during model training. Its purpose was to create more variation from the original training images, instead of collecting additional PCB images^[15]. This is useful in this project because the number of training images is not large, and the selected defects are small. For mouse bite and short defects, the target regions may be affected by background circuit patterns, so the model needs to learn defect features under slightly different image conditions. By changing the appearance of training images,

augmentation can reduce overfitting and help the model recognize similar defects in different situations^[29]. Previous studies have shown that suitable augmentation strategies can improve detection performance, especially when the dataset is small or the objects are difficult to detect^[15, 30].

YOLOv8 already includes several augmentation operations. These include geometric changes, such as flipping, scaling, and cropping, as well as color changes, such as hue, saturation, and brightness adjustment. Mosaic augmentation is also included in the default training process. In addition to augmentation, CLAHE-based enhancement methods have also been used to improve image contrast and reduce noise in low-quality images^[31]. In this project, the default YOLOv8 augmentation setting was used for G2, G4, and G6. For G1 and G3, augmentation was turned off through the `no_aug.yaml` configuration file. G5 did not train a new model. It used the trained G2 model and tested different CLAHE settings during inference. This design made it easier to compare the effect of data augmentation with the effect of CLAHE preprocessing.

2. Contrast Limited Adaptive Histogram Equalization (CLAHE)

Contrast Limited Adaptive Histogram Equalization, or CLAHE, was used as the contrast enhancement method in this project. It has been used in industrial vision inspection and medical image processing, mainly because it can improve local contrast without simply increasing the contrast of the whole image^[32]. In CLAHE, an image is first divided into several small local regions, usually called tiles. Histogram equalization is then applied inside each tile. During this process, a parameter called `clipLimit` is used to limit the height of the histogram, so that the enhancement does not become too strong^[33]. The clipped part is redistributed across the gray-level range. In this way, local contrast can be improved while noise amplification is controlled to some extent^[34].

What matters for this project is that CLAHE can make local details clearer while limiting the growth of noise^[35]. This is different from global histogram equalization, which applies one transformation to the whole image. This is different from global histogram equalization,

which applies one transformation to the whole image. It is also different from ordinary adaptive histogram equalization, because CLAHE adds a contrast limit to reduce over-enhancement^[31]. Public PCB image datasets have also been introduced for computer-vision-based PCB analysis, which shows that image data quality and image processing are important parts of PCB vision tasks^[36]. This feature is relevant to PCB defect detection. Mouse bite and short defects are often small and low in contrast. Some enhancement may make the defect regions easier to see, but too much enhancement may also strengthen background texture and make non-defect areas look more like defects^[7].

The effect of CLAHE depends strongly on its parameter setting. In this project, the two parameters considered were clipLimit and tileGridSize. The clipLimit controls how strong the contrast enhancement can be, while tileGridSize controls the size of the local regions used for histogram equalization^[31]. A larger clipLimit may produce clearer contrast, but it may also increase noise or make PCB background texture more obvious. A smaller tile size gives stronger local adjustment, but it can also increase computation and may make the enhanced texture less stable.

In this project, CLAHE was applied in the LAB color space. The RGB image was first converted into LAB format. CLAHE was then applied only to the L-channel, which represents luminance, while the A-channel and B-channel were kept unchanged. After enhancement, the image was converted back to RGB format. This setting was used because it changes brightness and contrast while avoiding obvious changes in color information.

For the mouse bite and short defect images used in this project, several CLAHE parameter combinations were tested. The tested clipLimit values ranged from 1.0 to 5.0, and the tested tileGridSize values ranged from (4,4) to (16,16). We compared the detection results under these settings to select a more suitable CLAHE configuration for the current PCB defect dataset.

2.1.3 Experimental Group Design

2.1.3.1 Experimental group design presentation

In this part of the experiment, six groups were set up and named G1 to G6. The main difference among these groups was how the images were processed during training and inference. Some groups used YOLOv8 default data augmentation, some used CLAHE only during inference, and one group used both original and CLAHE-enhanced images for training. The CLAHE parameter setting was also different among the related groups. Table 2.2 lists the settings used in each group. To keep the comparison fair, the same YOLOv8n model was used in all groups. The main training settings were also kept unchanged, including 80 epochs, a batch size of 16, and an input image size of 640×640 pixels. All experiments were carried out under the same hardware condition.

Group	Training Set Composition	Training Augmentation	Inference CLAHE	CLAHE Parameters
G1	Original images	Disabled	Not applied	—
G2	Original images	Enabled	Not applied	—
G3	Original images	Disabled	Applied	Default (clipLimit=2.0, tileGridSize= (8,8))
G4	Original images	Enabled	Applied	Default (clipLimit=2.0, tileGridSize= (8,8))

G5	Original images	Enabled	Applied	Optimal (clipLimit=1.0, tileGridSize=(6,6))
G6	Original + optimal CLAHE- enhanced images	Enabled	Applied	Optimal (clipLimit=1.0, tileGridSize=(6,6))

Table 2.2 Overview of experimental group configurations

2.1.3.2 Experimental group design concept

The six experimental groups were arranged in a step-by-step way. The purpose was to separate the effects of data augmentation, CLAHE preprocessing, CLAHE parameter selection, and training-inference preprocessing consistency, rather than testing all factors at the same time.

(1) Baseline Effect of Training Augmentation (G1 vs. G2)

G1 and G2 both used the original training images. The difference was that G1 was trained without data augmentation, while G2 used the default augmentation settings in YOLOv8. By comparing these two groups, the effect of training-stage augmentation on the detection performance of YOLOv8n can be observed.

(2) Domain Shift Effect of CLAHE Without Augmentation (G3 vs. G1)

G3 was based on the model setting of G1, but default CLAHE preprocessing was applied during inference. This comparison was used to check whether applying CLAHE only at the inference stage would change the input image style and lead to a performance drop when the model had not seen CLAHE-enhanced images during training.

(3) Domain Shift Effect of CLAHE With Augmentation (G4 vs. G2)

G4 was based on the model setting of G2, with default CLAHE preprocessing added during inference. By comparing G4 with G2, this experiment examined whether the model trained with data augmentation could still maintain stable performance when the test images were changed by CLAHE preprocessing.

(4) Mitigation Effect of CLAHE Parameter Optimization (G5 vs. G2)

G5 used the trained G2 model and applied the optimal CLAHE parameters during inference, with $\text{clipLimit} = 1.0$ and $\text{tileGridSize} = (6,6)$. These parameters were identified through grid search in this study. The comparison between G5 and G2 was used to evaluate whether CLAHE parameter optimization could reduce the performance loss caused by the difference between training and inference images.

(5) Effect of the Training-Inference Consistency Strategy (G6 vs. G5)

G6 used a mixed training set consisting of the original training images and the images processed with the optimal CLAHE parameters. After this expansion, the training set contained 294 images in total. During inference, the same optimal CLAHE parameters were applied to the validation images. By comparing G6 with G5 on the optimal CLAHE-processed validation set, this experiment evaluates whether introducing the expected preprocessing style into the training stage can reduce the mismatch between training and inference images. It also examines whether this consistency strategy can achieve better detection performance than only adjusting CLAHE parameters during inference.

2.1.3.3 Theoretical Basis of the G6 Mixed Training Strategy

Among the six experimental groups, G6 used a different training set design from the other groups. In this group, the original training images were mixed with the images enhanced by the optimal CLAHE parameters, namely $\text{clipLimit} = 1.0$ and $\text{tileGridSize} = (6,6)$. As a result, the training set was expanded to 294 images. This design was not only used to increase the number of training images, but also to make the training images closer to the

images used during CLAHE-based inference. The reason for using this mixed training strategy can be explained from the following aspects.

(1) As an Extension of Data Augmentation

From the perspective of data augmentation, the mixed training strategy used in G6 can be regarded as an extension of ordinary augmentation methods. The main purpose of data augmentation is to increase the diversity of training samples by applying different transformations to the original images, so that the model can learn more general features^[15]. Common augmentation operations, such as random flipping, scaling, and color adjustment, change the image appearance while keeping the defect labels unchanged. CLAHE enhancement also changes the visual appearance of the image without changing the defect category or the bounding box position. Therefore, adding CLAHE-enhanced images into the training set can be understood as a form of image-level augmentation, although it is applied in a fixed and deterministic way rather than randomly during training^[29].

Similar ideas have also been used in other object detection tasks. For example, in UAV-based agricultural detection, Pix2Pix generative adversarial networks were used to generate additional training images, and these generated images were mixed with real images for YOLOv8 training. The study reported a 5.6% increase in mAP50 after using this mixed training strategy^[37]. Although the image generation method in that study is different from CLAHE enhancement, both approaches share a similar idea: transformed images can be added to the training set to increase data variation and improve model learning. This provides methodological support for the design of G6 in this study.

(2) Domain Shift Perspective: Reducing Training-Inference Distribution Mismatch

Domain shift refers to the situation where the data distribution during testing is different from that during training, which may lead to a decline in model performance^[24]. In practical deep learning applications, this problem is common because the images used in deployment are often not exactly the same as the images used for training.

In this project, this problem has a specific cause. If the model is trained only with original PCB images, but the images are processed by CLAHE during inference, the model is tested on images with a different contrast and texture style. The defect labels are still the same, but the visual appearance of the input image has changed. This may make the model less accurate, especially for small defects whose features are already weak.

Previous research has discussed the value of keeping transformations more consistent between training and testing when dealing with domain shift^[38]. Gao et al. discussed that data augmentation pairs and consistency regularization can help improve generalization under domain shift conditions^[38]. G6 was designed based on a similar idea. In G6, the model is not trained only on original images. It also sees CLAHE-enhanced images during training. Therefore, when CLAHE is used again during inference, the input style is no longer completely new to the model. This is the main reason why G6 was designed to reduce the mismatch between training and inference images.

(3) Performance Gains Driven by Data Diversity

The mixed training strategy may also bring another benefit: the training set becomes more varied. In G6, each original training image keeps its original label and bounding box, and a CLAHE-enhanced version is added as another training sample. These two image versions describe the same defect location, but they do not look exactly the same. Their contrast, brightness distribution, and local details are slightly different. In this way, the model can learn the same defect from more than one image appearance.

This is useful for the current dataset because the number of training images is limited and the selected defect types are small. If the model only learns from one image style, it may depend too much on the contrast and texture pattern of the original images. Adding CLAHE-enhanced images gives the model more visual variation during training, which may help it make more stable predictions on the test set^[13].

CLAHE-based methods have been widely used for image enhancement, and some studies also show that enhanced images can be used together with detection models^[32, 34]. Therefore, in this project, CLAHE-enhanced images were added into the training set while YOLOv8 default augmentation was still enabled. Although that study was not about PCB defect detection, it supports the idea that CLAHE enhancement and data augmentation can be combined rather than treated as two separate choices. Based on this consideration, G6 adds CLAHE-enhanced images into the training set while keeping YOLOv8 default data augmentation enabled.

In summary, the G6 mixed training strategy was designed for three reasons. First, adding CLAHE-enhanced images can be treated as an image-level extension of data augmentation. Second, it helps reduce the difference between the image style used in training and the image style used during CLAHE-based inference. Third, it increases the visual diversity of the training set. During this process, the validation set and test set were not used to generate training samples, and the original separation among training, validation, and test data was kept unchanged. Therefore, this design does not introduce information leakage.

2.1.4 Model Configuration

This study used YOLOv8n, namely You Only Look Once version 8 nano, as the detection model. YOLOv8 has been widely used in industrial defect detection because it is relatively lightweight and can maintain a reasonable balance between detection accuracy and processing speed^[20]. The nano version was used because this project did not require a large detection model. YOLOv8n has about 3.2 million parameters, so it was more suitable for the available computing resources. Another reason was that YOLOv8 already includes default data augmentation and training settings. This made it easier to test preprocessing strategies without changing the model structure. Previous studies have also reported that YOLOv8-based models can perform well in PCB micro-defect detection tasks^[21].

For the groups that required training, the same basic training settings were used. Each model was trained for 80 epochs with a batch size of 16. The input image size was fixed at 640×640 pixels. Cosine learning rate scheduling was enabled by setting `cos_lr=True`, and the early stopping patience was set to 20 epochs. The optimizer used the default SGD setting in YOLOv8, with an initial learning rate of 0.01, momentum of 0.937, and weight decay of 0.0005. These settings were kept the same across the training groups so that the differences in results mainly came from the preprocessing design rather than from changes in the training configuration.

Data augmentation was controlled according to the experimental group design. G2, G4, and G6 were trained with the default augmentation setting in YOLOv8, including Mosaic augmentation, random flipping, random scaling, random cropping, and color-related changes such as hue, saturation, and brightness adjustment. G1 and G3 were trained with augmentation turned off through the custom `no_aug.yaml` configuration file. G5 did not train another model. It used the trained G2 model to test different CLAHE parameter combinations during inference. In this way, the effect of data augmentation and the effect of CLAHE preprocessing could be compared under a clearer experimental setting.

Because of GPU compatibility problems, all experiments were run on CPU for both training and inference. Training on CPU took more time than GPU training, but the same hardware condition was used for every group. Therefore, the comparison among G1 to G6 was not affected by differences in computing hardware. The performance differences mainly reflected the different preprocessing strategies used in each group.

2.1.5 Evaluation Metrics

The detection results of each experimental group were evaluated mainly by mean Average Precision, namely mAP. Since this project compares object detection models, the evaluation considers two points: whether the defect is found, and whether the predicted

box is close to the labelled defect region. For this reason, two mAP-related metrics were used in the experiment: mAP50 and mAP50-95.

(1) mAP50

mAP50 is calculated when the Intersection over Union (IoU) threshold is set to 0.5. In this case, a predicted bounding box is counted as correct if its overlap with the ground-truth box reaches 0.5. This metric gives a less strict requirement for localization, so it is useful for checking the general detection ability of the model. In PCB defect detection studies, mAP50 is also commonly reported when comparing YOLO-based models^[21].

(2) mAP50-95

mAP50-95 uses several IoU thresholds instead of only one threshold. In this study, the thresholds ranged from 0.5 to 0.95 with a step of 0.05. This metric is stricter than mAP50 because the predicted box must match the labelled box more closely at higher IoU thresholds. Therefore, mAP50-95 helps show whether the detector can locate defects accurately, not only whether it can find them. Similar AP-based evaluation metrics are also used in recent PCB defect detection studies^[29].

In addition to the overall mAP values, the AP of each defect category was also recorded. The two categories in this project were mouse bite and short defects. Reporting class-level AP makes it easier to see whether the same preprocessing strategy affects the two defect types differently. This is necessary because mouse bite defects are usually smaller and have weaker boundaries than short defects. Their AP values may therefore change more clearly when preprocessing methods are changed^[21].

The metrics were calculated on both the validation set and the test set. The validation set was used during model comparison and CLAHE parameter selection. The test set was kept for the final evaluation only. It was not used for training, model selection, or parameter tuning. Therefore, the test-set results were used to evaluate how well each model performed on unseen PCB images.

2.2 Results

2.2.1 Overall Performance Comparison

The same independent test set was used to compare all experimental groups. This set contained 47 images and 199 annotated defect instances. It was not used for training, model selection, or CLAHE parameter search. Therefore, the test-set results were used as the final performance comparison among G1 to G6. Table 2.3 lists the main results, including mAP50, mAP50-95, and the AP values for mouse bite defects and short defects.

Group	mAP50	mAP50-95	AP_mouse_bite	AP_short
G1	0.797	0.365	0.755	0.838
G2	0.907	0.432	0.855	0.959
G3	0.723	0.301	0.669	0.776
G4	0.840	0.386	0.755	0.925
G5	0.895	0.422	0.841	0.949
G6	0.910	0.436	0.877	0.942

Table 2.3 Overall performance comparison on the independent test set

As shown in Table 2.3, the six experimental groups produced different detection results under different preprocessing settings. Figure 2.1 further shows the mAP50 values of each group. G1 was the baseline group. It was trained and tested without data augmentation or CLAHE preprocessing, and its mAP50 was 0.797. When YOLOv8 default data augmentation was used in G2, the mAP50 increased to 0.907. The absolute increase was 0.110, or 13.8% in relative terms. Since the training images and inference images were otherwise kept in the original style, this comparison shows that augmentation was helpful for this dataset.

When default CLAHE parameters were applied only during inference, the model performance decreased. G3, which was trained without augmentation, achieved an mAP50 of 0.723, while G4, which was trained with augmentation, achieved an mAP50 of 0.840. Compared with G1 and G2, both groups showed lower results after default CLAHE processing. This suggests that changing the image style during inference may introduce a mismatch between the training and test images. After the optimal CLAHE parameters were identified through grid search, G5 used the trained G2 model and achieved an mAP50 of 0.895 on the optimal CLAHE-processed test set. This result was higher than those of G3 and G4, but still lower than the G2 result obtained on original images. Therefore, CLAHE parameter optimization during inference can reduce part of the performance loss, but it does not fully solve the mismatch caused by inconsistent preprocessing.

G6 used a mixed training set consisting of original images and optimal CLAHE-enhanced images, and the same optimal CLAHE parameters were also applied during inference. This group achieved the highest mAP50 among all groups, reaching 0.910. Compared with G5, the improvement was 1.7%, and the result was also slightly higher than that of G2 on original images. This indicates that including the target preprocessing style in the training stage can help the model adapt better to CLAHE-processed images, while the expanded training data may also bring additional benefits.

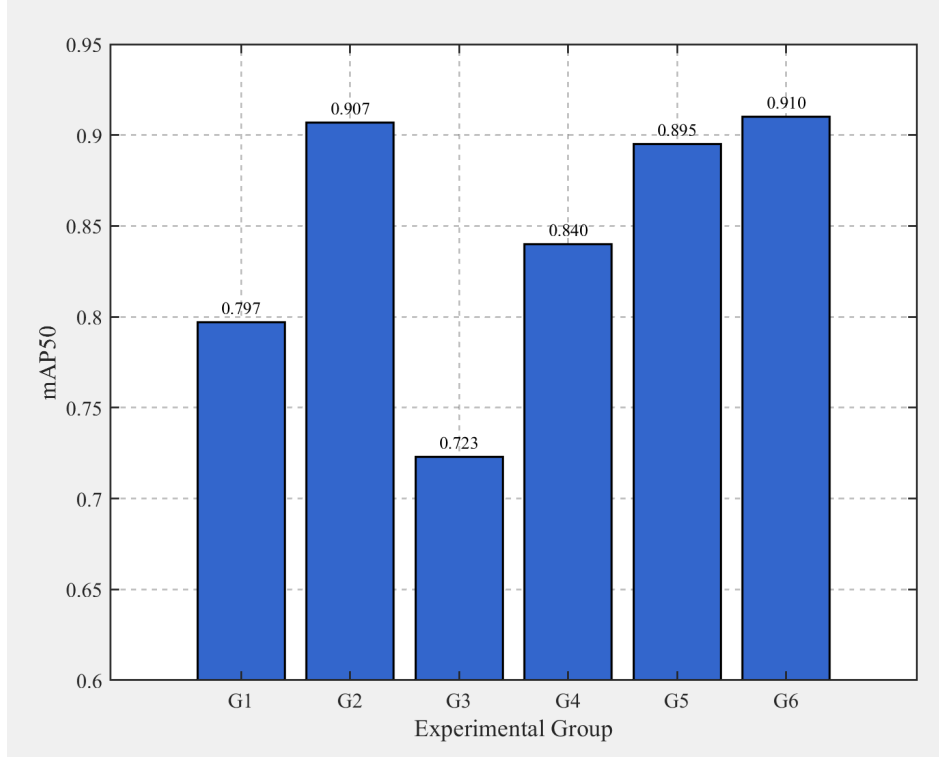


Figure 2.1 Bar chart comparing mAP50 among experimental groups

To compare the detection results for different defect categories, Figure 2.2 shows the AP values for mouse bite defects and short defects. The two defect types generally follow the same trend as the overall mAP50 results, but mouse bite defects changed more clearly when the preprocessing setting was changed. For example, in G3, default CLAHE was applied during inference without training augmentation. In this case, the AP for mouse bite defects decreased from 0.755 in G1 to 0.669, a decrease of 11.4%. For short defects, the AP decreased from 0.838 to 0.776, a decrease of 7.4%. This difference is reasonable because mouse bite defects are smaller and less obvious in the image. When CLAHE enhances background texture or noise together with the defect region, the model may have more difficulty separating the defect from the surrounding circuit pattern.

After using the optimal CLAHE parameters, G5 achieved a mouse bite AP of 0.841. This result was better than the default CLAHE groups, but it was still lower than the mouse bite AP of G2, which was 0.855. In contrast, G6 achieved a mouse bite AP of 0.877, which was

higher than both G5 and G2. This result indicates that the training-inference consistency strategy was especially useful for mouse bite defects, because these defects depend more strongly on stable and clear local feature representation.

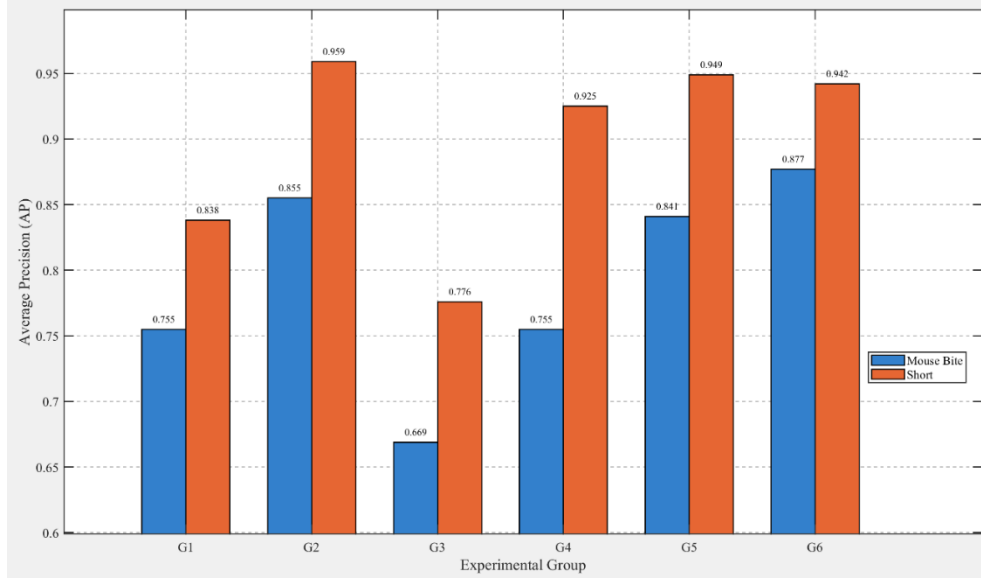


Figure 2.2 Grouped bar chart comparing AP for mouse bite defects and short defects

Overall, the results in this subsection show several patterns. First, data augmentation improved the baseline performance of YOLOv8n. Second, directly applying default CLAHE during inference reduced detection performance, suggesting that inconsistent preprocessing between training and inference may cause a distribution mismatch. Third, optimizing the CLAHE parameters reduced part of this performance loss, but the result of G5 still did not exceed G2 on original images. Finally, G6 achieved the best overall result by adding optimal CLAHE-enhanced images to the training set and using the same optimal CLAHE parameters during inference. In this group, the model had already seen CLAHE-processed images during training, so the inference images were closer to part of the training distribution. The mixed training set also gave the model more image variation. The following subsections discuss each comparison in more detail.

2.2.2 Baseline Effect of Training Augmentation (G1 vs G2)

G1 and G2 were used to check the effect of training-stage data augmentation. The two groups used the same original training images, and neither group used CLAHE during inference. The only difference was the augmentation setting during training. G1 was trained without augmentation, while G2 was trained with the default augmentation setting in YOLOv8. Therefore, the comparison between these two groups mainly reflects the effect of data augmentation.

Table 2.4 shows the test-set results of G1 and G2. G2 performed better than G1 in all reported metrics. The mAP50 increased from 0.797 to 0.907. This is an absolute increase of 0.110, corresponding to a relative improvement of 13.8%. Since the model structure, dataset split, and inference images were kept the same, this improvement can mainly be linked to the use of data augmentation during training. Figure 2.3 shows the mAP50 comparison between G1 and G2.

Metric	G1	G2	Absolute Improvement	Relative Improvement
mAP50	0.797	0.907	+0.110	+13.8%
mAP50-95	0.365	0.432	+0.067	+18.4%
AP_mouse_bite	0.755	0.855	+0.100	+13.2%
AP_short	0.838	0.959	+0.121	+14.4%

Table 2.4 Performance comparison between G1 and G2

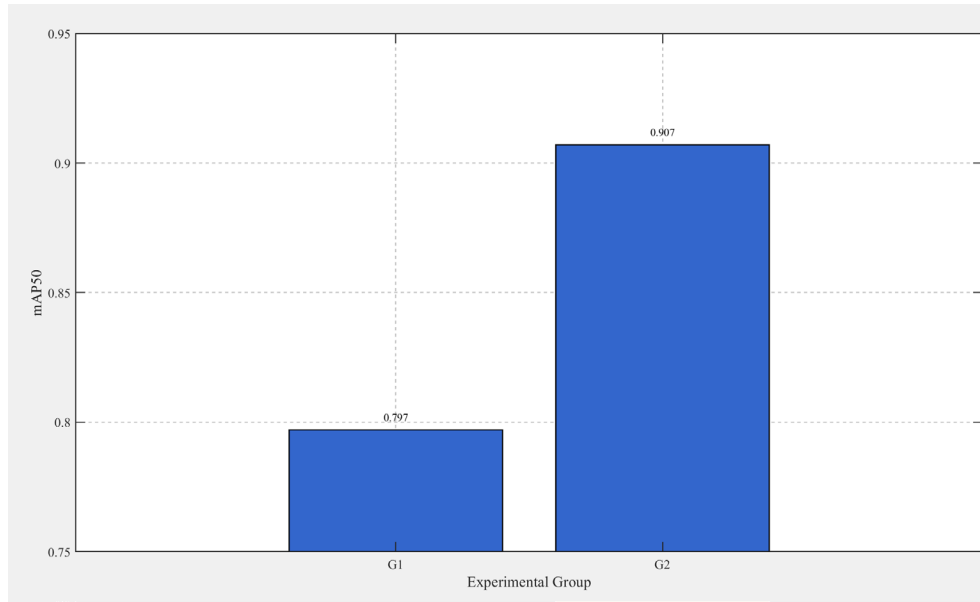


Figure 2.3 Bar chart comparing mAP50 between G1 and G2

The AP values of the two defect categories also increased after augmentation was enabled. As shown in Figure 2.4, the AP for mouse bite defects increased by 13.2%, and the AP for short defects increased by 14.4%. This means that the improvement was not limited to a single defect type. For mouse bite defects, this improvement is particularly valuable because these defects are small and their boundaries are not always clear. More variation in the training images may help the model learn these weak features more reliably.

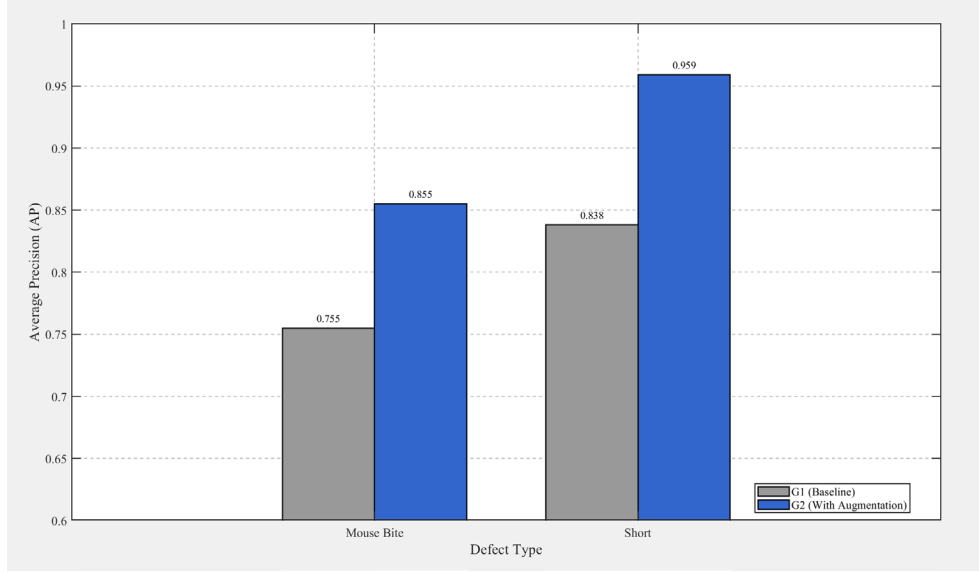


Figure 2.4 Bar chart comparing AP for mouse bite defects and short defects between G1 and G2

These results are consistent with previous studies showing that data augmentation can benefit small-target detection^[21, 29]. In this study, YOLOv8's built-in augmentation operations, such as Mosaic augmentation, random scaling, and random flipping, increased the visual variation of the training images. This may have helped the model learn more stable feature representations from a limited number of training samples. For mouse bite defects, the added image variation is useful because the defect regions are small, weak in contrast, and easy to confuse with nearby circuit patterns.

2.2.3 Domain Shift Effect of CLAHE (G3 vs G1, G4 vs G2)

This subsection examines the effect of applying CLAHE only during inference. Two comparisons were made. The first comparison is between G3 and G1, where both models were trained without data augmentation, but G3 used default CLAHE preprocessing during inference. The second comparison is between G4 and G2, where both models were trained with data augmentation, but only G4 used default CLAHE during inference. Through these

comparisons, the influence of inference-stage CLAHE can be observed under both non-augmented and augmented training conditions.

Table 2.5 summarizes the results of G1, G2, G3, and G4 on the test set. The results show that directly applying default CLAHE during inference reduced model performance in both comparison groups. This means that the performance decrease was observed regardless of whether data augmentation was used during training.

Comparison	Metric	Original Performance	CLAHE Performance	Absolute Drop	Relative Drop
G3 vs G1	mAP50	0.797	0.723	-0.074	-9.3%
G3 vs G1	AP_MB	0.755	0.669	-0.086	-11.4%
G3 vs G1	AP_SC	0.838	0.776	-0.062	-7.4%
G4 vs G2	mAP50	0.907	0.840	-0.067	-7.4%
G4 vs G2	AP_MB	0.855	0.755	-0.100	-11.7%
G4 vs G2	AP_SC	0.959	0.925	-0.034	-3.5%

Table 2.5 Performance changes caused by default CLAHE during inference

Figure 2.5 compares the mAP50 values of the four groups. G3 achieved an mAP50 of 0.723, which was 9.3% lower than G1 with an mAP50 of 0.797. Similarly, G4 achieved an mAP50 of 0.840, which was 7.4% lower than G2 with an mAP50 of 0.907. These results suggest that applying default CLAHE only during inference changed the input image distribution and caused a mismatch with the images used for training.

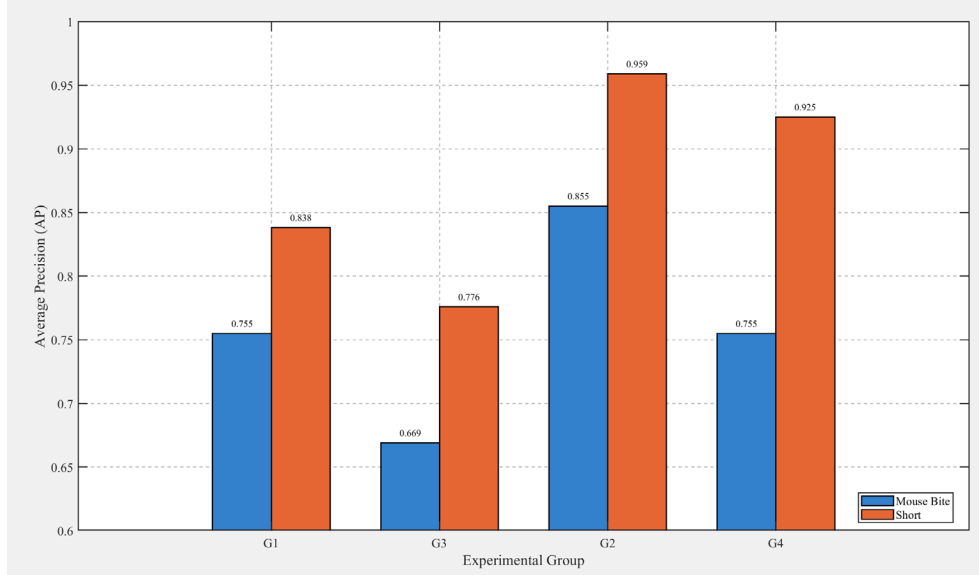


Figure 2.5 Bar chart comparing mAP50 among G1, G3, G2, and G4

The AP values for the two defect categories are further compared in Figure 2.6. Mouse bite defects were more affected by this preprocessing mismatch than short defects. Under the no-augmentation condition, namely G3 compared with G1, the AP for mouse bite defects decreased by 11.4%, while the AP for short defects decreased by 7.4%. Under the augmentation condition, namely G4 compared with G2, the AP for mouse bite defects decreased by 11.7%, whereas the AP for short defects decreased by only 3.5%. This result indicates that mouse bite defects are more sensitive to changes in image contrast and texture. Since these defects are small and have relatively unclear boundaries, inappropriate contrast enhancement may strengthen background texture or noise and make the defect features less distinguishable.

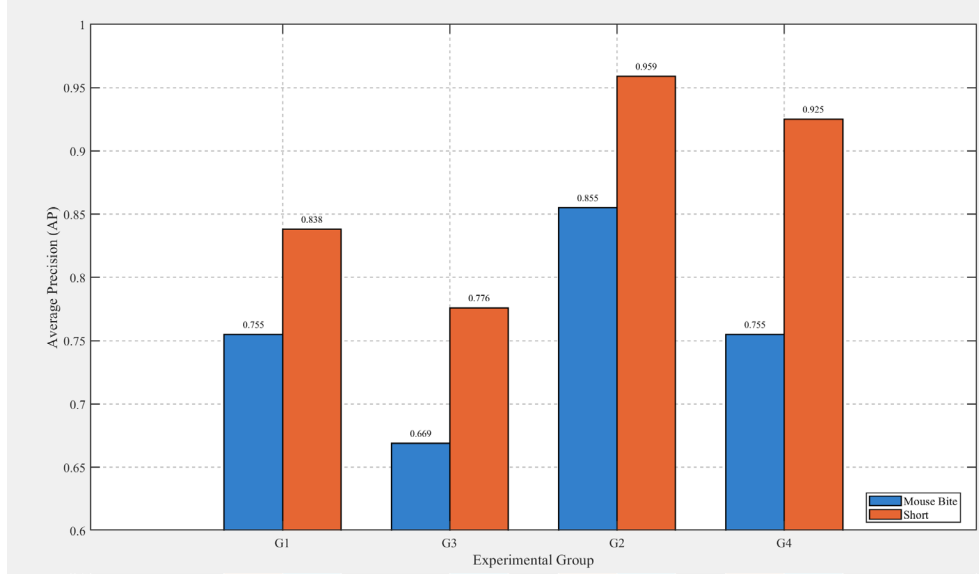


Figure 2.6 Grouped bar chart comparing AP for mouse bite defects and short defects among G1, G3, G2, and G4

It should also be noted that G4 achieved a higher mAP50 than G3, with values of 0.840 and 0.723, respectively. This suggests that training-stage data augmentation provided some robustness when the inference images were changed by CLAHE. Even so, G4 was still 7.4% lower than G2. In other words, augmentation improved the model to some extent, but it did not make the model fully comfortable with the CLAHE-processed images. The drop from G2 to G4 shows that the difference between the training images and the inference images still affected the final detection result.

This result can be explained by domain shift^[24]. In this experiment, the models mainly learned from original PCB images during training. During inference, CLAHE changed the contrast and local texture of the input images. Although the defect labels were unchanged, the images seen by the model no longer looked exactly like the images used for training. This difference is likely one reason why G3 and G4 performed worse than their corresponding original-image groups.

2.2.4 Mitigation Effect of Parameter Optimization (G5 vs G2)

The results in the previous sections show that applying CLAHE with default parameters only during inference reduced the detection performance of both G3 and G4. Although the model trained with data augmentation showed better tolerance than the model without augmentation, the performance of G4 was still lower than that of G2. This means that training augmentation alone could not fully solve the mismatch caused by the change in preprocessing style. Therefore, it is necessary to further examine whether the performance loss can be reduced by adjusting the CLAHE parameters used during inference.

For this purpose, experimental Group G5 was designed. G5 used the trained G2 model, which was trained on original images with YOLOv8 default data augmentation. During inference, however, the test images were processed using the optimal CLAHE parameters instead of the default CLAHE parameters. The optimal parameter combination was determined by grid search on the validation set. Since the model itself was fixed in this experiment, the comparison mainly focused on whether changing the CLAHE preprocessing parameters could make the enhanced images more suitable for the trained model.

2.2.4.1 Determination of Parameter Search Range and Step Size

The effect of CLAHE depends mainly on two parameters: the contrast limit threshold, namely clipLimit, and the tile grid size, namely tileGridSize. The search range and step size of these two parameters need to be set carefully, because they affect both the enhancement result and the computational cost of the parameter search.

Determination of the Contrast Limit Threshold (clipLimit):

The clipLimit parameter controls the maximum strength of local contrast enhancement in CLAHE. If this value is too low, the enhancement may be weak and the defect region may still be difficult to see. If it is too high, background texture and noise may also be enhanced. Previous studies on CLAHE-based image enhancement have shown that the setting of

contrast limitation can affect detail preservation, noise amplification, and over-enhancement artifacts^[30-33]. Therefore, the clipLimit value needs to be tested rather than fixed only by the default setting.

In this study, seven clipLimit values were tested: 1.0, 1.5, 2.0, 2.5, 3.0, 4.0, and 5.0. The range from 1.0 to 5.0 was selected to cover mild, medium, and relatively strong contrast enhancement. The values 1.5 and 2.5 were added because the best result might appear in the lower enhancement range, rather than exactly at an integer value. This design also avoided using extremely high enhancement settings, which may increase the risk of noise amplification. The main purpose of this search was to find a suitable enhancement strength for mouse bite and short defects in the current PCB dataset.

Determination of the Tile Grid Size (tileGridSize):

The tileGridSize parameter determines how the image is divided into local regions before histogram equalization. A smaller grid gives stronger local adjustment, while a larger grid makes the enhancement closer to a more global contrast adjustment. In image enhancement studies, tile or window size is often treated as an important CLAHE parameter because it affects the balance between local detail enhancement and image stability^[34, 35].

In this study, the input images were resized to 640×640 pixels, while mouse bite and short defects occupied only small regions in the image. Based on this condition, six tileGridSize values were tested: (4,4), (6,6), (8,8), (10,10), (12,12), and (16,16). The (8,8) setting was included because it is a common default choice in CLAHE implementation. The smaller settings, (4,4) and (6,6), were used to check whether finer local partitioning was more suitable for small PCB defects. The larger settings, (10,10), (12,12), and (16,16), were included to compare the effect of weaker local adjustment. The (6,6) setting was added to explore an intermediate division between (4,4) and (8,8), where the enhancement of small local defect features might change more obviously.

2.2.4.2 Analysis of Parameter Grid Search Results

Table 2.6 compares the test-set results of G5, G2, and G4. G5 represents inference using the optimal CLAHE parameters, G2 represents inference on original images without CLAHE, and G4 represents inference using the default CLAHE parameters. Figure 2.7 shows the heatmap obtained from the CLAHE parameter grid search. The heatmap was drawn using the mAP50 values of different parameter combinations on the validation set, so it can be used to observe how clipLimit and tileGridSize affected detection performance.

Group	mAP50	mAP50-95	AP_mouse _bite	AP_short
G2 (Original image inference)	0.907	0.432	0.855	0.959
G4 (Default CLAHE inference)	0.840	0.386	0.755	0.925
G5 (Optimal CLAHE inference)	0.895	0.422	0.841	0.949
G5 vs G2	-1.2%	-2.3%	-1.6%	-1.0%
G5 vs G4	+6.5%	+9.3%	+11.4%	+2.6%

Table 2.6 Performance comparison of G5 with G2 and G4

As shown in Table 2.6, G5 achieved an mAP50 of 0.895 on the test set. This value was close to the result of G2, which was evaluated on original images, with only a small gap of about 1.2%. Compared with G4, which used default CLAHE parameters and obtained an mAP50 of 0.840, G5 showed a relative improvement of 6.5%. This result suggests that, for the current model and dataset, CLAHE parameter selection has a clear influence on detection performance. Using the optimal CLAHE parameters reduced much of the performance loss caused by applying CLAHE during inference, while the default parameters were not suitable for this task.

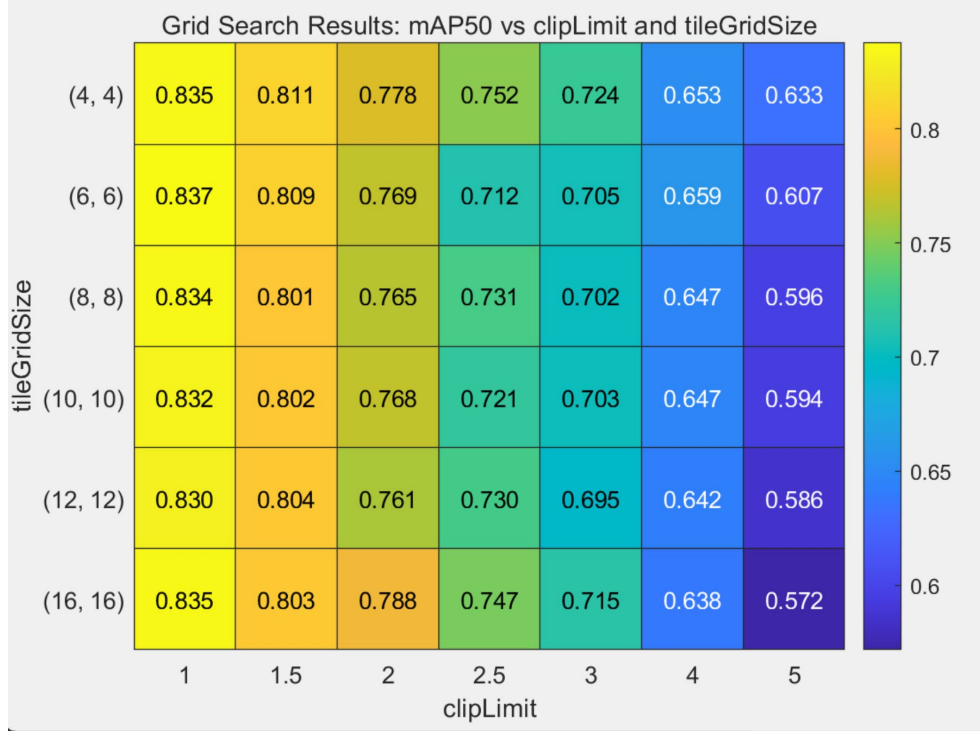


Figure 2.7 Heatmap of CLAHE parameter grid search based on G2 model mAP50 on the validation set

As shown in Figure 2.7, the best parameter combination appeared in the low-clipLimit and medium-grid-size region. Specifically, the optimal CLAHE parameters were clipLimit = 1.0 and tileGridSize = (6,6), with an mAP50 of 0.837 on the validation set. Other relatively good results were mainly found when clipLimit was between 1.0 and 2.0. When clipLimit continued to increase, the detection performance generally decreased. This result indicates that PCB defect images are sensitive to the strength of contrast enhancement. A lower clipLimit can improve the visibility of small defect regions to some extent, but excessive enhancement may also strengthen background texture and noise. For mouse bite and short defects, whose visual features are relatively weak, this can make the defect boundaries less clear and reduce the model's ability to distinguish defects from the background^[21, 31].

2.2.4.3 Theoretical Explanation of Parameter Sensitivity

Advantages of Low Contrast Limit Parameters (clipLimit $\leq$ 2.0):

When clipLimit was set to a low value, CLAHE applied a weaker local contrast enhancement. This was useful for the PCB images in this study because the background was not completely smooth. Copper foil patterns, solder mask texture, and slight unevenness on the substrate could also be enhanced together with the defect region. In the PCB images used in this study, if these background details became too strong, they could look similar to small defects and make the detector harder to evaluate.

The heatmap results show that the better mAP50 values were mainly found when clipLimit was 1.0 or 2.0. This means that, for the current dataset, mild enhancement was more suitable than strong enhancement. Related image enhancement studies also show that CLAHE parameters, especially the contrast limit, can affect the balance between detail enhancement and noise control^[31, 32, 35]. For mouse bite defects, this point is important. A low contrast limit may help make the notch edge clearer, but it does not strongly amplify the surrounding circuit texture. In comparison, stronger enhancement may reduce the difference between the true defect edge and the enhanced background^[21].

Negative Impact of High Contrast Limit Parameters (clipLimit $\geq$ 3.0):

When clipLimit was increased, CLAHE gave stronger histogram stretching within local regions. This did not only affect defect-related details. It also strengthened weak background variations in the PCB image. Previous CLAHE studies have noted that unsuitable enhancement settings may lead to artifacts or over-enhancement, especially when the enhancement strength is not adapted to the image content^[32].

In this experiment, the negative effect became clearer when clipLimit reached 3.0 or higher. Non-defect regions, such as small scratches, oxidation marks, or background texture, could become more visible after enhancement. At the same time, real defect features could become less distinctive because the nearby background was also strengthened. This was especially relevant in this project, because mouse bite defects often appeared as small edge notches, while short defects appeared as narrow bridge-like connections in the selected images. In the heatmap, the results under clipLimit = 5.0 were generally lower than those

under lower clipLimit settings. Therefore, strong contrast enhancement was not suitable for this PCB defect detection task.

Mechanism of Tile Grid Size (tileGridSize) Influence:

Compared with clipLimit, tileGridSize had a milder effect on detection performance, but the results still showed that the grid size should not be ignored. A smaller tileGridSize, such as (4,4), divides the image into finer local regions. This can make local adjustment stronger, but the histogram of each region is based on a smaller part of the image. If the local region is too small, the enhancement may become less stable. A larger tileGridSize, such as (16,16), makes the enhancement closer to a broader contrast adjustment. This may reduce local instability, but it may also weaken the enhancement of very small defect details.

In this study, tileGridSize = (6,6) gave the best result among the tested settings. This suggests that it provided a better balance between local contrast adjustment and histogram stability for 640×640 PCB images. In this study, tileGridSize = (6,6) gave the best result among the tested settings. This suggests that it provided a better balance between local contrast adjustment and histogram stability for 640×640 PCB images. Related CLAHE-based detection studies also show that image enhancement settings can affect detection performance, especially when the enhanced image contains noise, halo effects, or over-enhanced local regions^[34].

2.2.4.4 Discussion on the Limitations of Parameter Optimization

Although CLAHE parameter optimization improved the result compared with the default CLAHE setting, it still did not make G5 outperform G2. Specifically, G5 improved mAP50 by 6.5% compared with G4, but its mAP50 was still slightly lower than that of G2 tested on original images, with values of 0.895 and 0.907, respectively. For mouse bite defects, the AP of G5 was 0.841, which was also lower than the AP of G2, namely 0.855. This result shows that optimizing CLAHE parameters during inference can recover part of the lost performance, but its effect is limited. The reason is that G5 only changes the input

images, while the model weights remain those learned from the original-image training distribution^[38].

Tang et al. discussed that distribution shift between the source domain and the target domain can reduce model performance, and test-time adaptation methods try to reduce this problem by adapting the model or feature representation during inference^[24]. In this project, G5 did not update the model parameters. It only changed the input images through CLAHE. Therefore, even though the optimal CLAHE parameters were better than the default setting, the CLAHE-processed images were still not exactly the same as the original training images. Wang et al. also discussed in their survey on online test-time adaptation that input-level adjustment is different from model-level adaptation. When the model parameters are not updated, the feature extraction process still mainly follows the patterns learned during training^[18]. In this study, even though the optimal CLAHE parameters made the enhanced images more suitable than the default setting, the visual distribution of CLAHE-processed images was still different from that of the original images. For example, the enhanced images usually had stronger local contrast and a different gray-level distribution. Since the convolutional weights of the G2 model were learned from original images, the model could not fully adapt to this changed image style during inference^[24].

Therefore, CLAHE parameter optimization can reduce the performance loss caused by inconsistent preprocessing to some extent, but it cannot completely solve the distribution mismatch between training and inference. This limitation leads to the next step of this study. Instead of only changing the inference images, the training stage can also be adjusted by allowing the model to learn from CLAHE-enhanced images. This is the reason for designing the training-inference preprocessing consistency strategy in G6, which is discussed in the following section.

2.2.5 Training-Inference Consistency Strategy (G6 vs G5, G6 vs G2)

The previous experiment with G5 showed that grid search optimization of CLAHE parameters could reduce part of the performance loss when the model was fixed. However, the mAP50 of G5 was 0.895, which was still lower than that of G2, namely 0.907, when G2 was tested on original images. This result suggests that inference-stage parameter optimization has its limitation. It only changes the input images to make them more suitable for the trained model, while the model weights remain unchanged. Therefore, the distribution difference between the training images and the CLAHE-processed inference images cannot be fully removed in this way^[24, 38]. Based on this result, this section further examines whether introducing the target preprocessing style during training can help the model adapt better to CLAHE-enhanced images.

To test this idea, experimental Group G6 was designed. The training set of G6 was composed of both original training images and images enhanced using the optimal CLAHE parameters identified in G5, namely clipLimit = 1.0 and tileGridSize = (6,6). After this expansion, the training set contained 294 images in total. During inference, the same optimal CLAHE parameters were applied to the test images. In this way, the preprocessing used during training and inference became more consistent. The model was therefore trained not only on original images, but also on images with the same enhancement style as the inference images. This design was expected to reduce the mismatch between training and inference distributions, while the enlarged training set could also provide more image variation.

Table 2.7 compares the test-set performance of G6, G5, and G2. G6 achieved an mAP50 of 0.910, which was higher than G5 with an mAP50 of 0.895 and also slightly higher than G2 with an mAP50 of 0.907. This shows that the training-inference consistency strategy performed better than inference-only parameter optimization in this experiment. For mouse bite defects, which were more difficult to detect, G6 achieved an AP of 0.877. This was 2.6% higher than G2 with an AP of 0.855 and 4.3% higher than G5 with an AP of 0.841.

These results suggest that adding optimal CLAHE-enhanced images into the training set was especially helpful for small and low-contrast defect detection.

Group	G2	G5	G6
Training Set Composition	Original images	Original images	Original + Optimal CLAHE
Training Augmentation	Enabled	Enabled	Enabled
Inference Preprocessing	None	Optimal CLAHE	Optimal CLAHE
mAP50	0.907	0.895	0.910
mAP50-95	0.432	0.422	0.436
AP_mouse_bite	0.855	0.841	0.877
AP_short	0.959	0.949	0.942

Table 2.7 Performance comparison of G6 with G5 and G2

Several observations can be made from Table 2.7.

First, the training-inference consistency strategy reduced the mismatch caused by applying CLAHE during inference. G6 was tested on optimal CLAHE-enhanced images and achieved an mAP50 of 0.910. This was 1.7% higher than G5, which used the same inference image type but did not include CLAHE-enhanced images during training. It was also slightly higher than G2, which achieved 0.907 on original test images. This result suggests that adding the target preprocessing style into the training phase helped the model become more familiar with the image distribution used during inference. As a result, the distribution gap between training and testing images was reduced compared with inference-only CLAHE parameter optimization^[24].

Second, the mixed training strategy also benefited from the increased diversity of the training data. In G6, the training set contained both original images and optimal CLAHE-enhanced images. The original images kept the natural appearance of the PCB samples, while the CLAHE-enhanced images introduced different contrast and local texture characteristics. This allowed the model to learn defect features under more than one image style, which may explain why G6 performed better than G5 and slightly better than G2 in terms of mAP50^[13,38]. The improvement in mouse bite AP was especially clear. Compared with G2, G6 increased the AP for mouse bite defects by 2.6%, suggesting that the additional image variation was useful for small and low-contrast defects.

Third, the results for short defects show that the effect of CLAHE-based preprocessing was not the same for all defect types. The AP for short defects was 0.942 in G6, lower than 0.959 in G2 and slightly lower than 0.949 in G5. This result shows that the consistency strategy did not improve every category equally. One possible reason is that short defects were already easier to identify in the original images. After CLAHE enhancement, some local contrast and texture changes may not add useful information for this defect type. They may even make the boundary between the short defect and the nearby circuit pattern less clear in some cases^[20]. However, this decrease was limited to the short defect AP. G6 still produced the highest overall mAP50 and the best AP for mouse bite defects, which were more difficult to detect in this experiment.

The G6 result gives a practical conclusion for this project. If CLAHE is used during inference, it is better not to add it only at the final testing stage. In G6, the training set already included images processed with the optimal CLAHE parameters, so the model had learned from both original images and CLAHE-enhanced images before testing. This reduced the difference between training and inference image styles. At the same time, the mixed training set provided more image variation. For PCB defect detection systems with a fixed preprocessing step, the preprocessing method should therefore be considered during training. This point is especially relevant when the target defects are small, low in contrast, or easily affected by local texture changes.

2.2.6 Visual Case Analysis

After the numerical comparison, several test images were checked to see how the models behaved in actual detection cases. This section uses four examples from the test set. The examples include both short defects and mouse bite defects, and the results of G2, G5, and G6 are compared. The selected images are not identical in difficulty: some contain clearer defects, while others have more complex circuit backgrounds or smaller defect regions. Therefore, they can help illustrate how the three strategies perform in specific image cases, in addition to the quantitative results reported above.

2.2.6.1 Short Defect Case: Three-Model Comparison

Figure 2.8 shows the detection results for the image 04_short_15. This image contains three short defect instances. All three models detected the three defects and the bounding boxes were generally located around the correct defect regions. The corresponding confidence scores are listed in Table 2.8.

Defect ID	G2 Confidence	G5 Confidence	G6 Confidence
1	0.89	0.87	0.90
2	0.72	0.72	0.79
3	0.70	0.62	0.81

Table 2.8 Confidence scores of G2, G5, and G6 for short defects in 04_short_15

From Table 2.8, G6 gave the highest confidence score for all three short defects. For Defect 3, the confidence score of G6 was 15.7% higher than that of G2 and 30.6% higher than that of G5. This result suggests that the mixed training strategy used in G6 helped the model respond more confidently to this short defect case. In comparison, although G5 used the optimal CLAHE parameters during inference, its confidence scores were less stable. For example, the confidence for Defect 3 decreased to 0.62. This may be because G5 still used

the model trained only on original images, so the model was not fully adapted to the CLAHE-enhanced image style. By contrast, G6 included optimal CLAHE-enhanced images during training and used the same preprocessing setting during inference. This made the training and testing image styles more consistent, which may explain its higher confidence scores in this example.

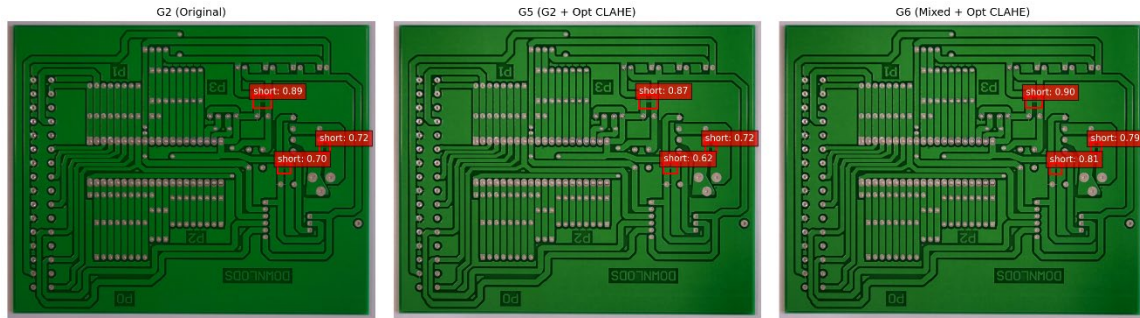


Figure 2.8 Detection results of G2, G5, and G6 for short defects in 04_short_15

2.2.6.2 Mouse Bite Defect Case: Six-Defect Image

Figure 2.9 shows the detection results for the image 05_mouse_bite_07. This image contains six mouse bite defect instances. All six defects were detected by G2, G5, and G6, and the predicted boxes were generally located around the corresponding defect regions. The confidence scores of the three models are listed in Table 2.9.

Defect ID	G2 Confidence	G5 Confidence	G6 Confidence
1	0.77	0.81	0.86
2	0.58	0.52	0.86
3	0.55	0.56	0.80
4	0.66	0.70	0.82
5	0.62	0.62	0.63
6	0.61	0.63	0.82

Table 2.9 Confidence scores of G2, G5, and G6 for mouse bite defects in 05_mouse_bite_07

As shown in Table 2.9, G6 produced the highest confidence score for all six mouse bite defects in this image. For Defect 2, the confidence score increased from 0.58 in G2 to 0.86 in G6, corresponding to an improvement of 48.3%. For Defect 6, the score increased from 0.63 in G5 to 0.82 in G6, with an improvement of 30.2%. These changes indicate that G6 gave a stronger response to several mouse bite defects that had relatively low confidence scores in the other two models.

The comparison also shows that G5 did not improve all defects consistently. For example, the confidence score of Defect 1 increased from 0.77 in G2 to 0.81 in G5, but the score of Defect 2 decreased from 0.58 to 0.52. This suggests that using the optimal CLAHE parameters only during inference may help in some defect regions, but it may also reduce confidence in other regions. One possible reason is that the G5 model was still trained only on original images, so it was not fully matched with the CLAHE-enhanced image style used during inference.

In contrast, G6 was trained with both original images and optimal CLAHE-enhanced images. Therefore, when the same optimal CLAHE parameters were applied during inference, the model had already learned from similar image appearances during training. This may explain why G6 showed higher confidence scores for all six mouse bite defects in this case, especially for those defects that were less confidently detected by G2 or G5.

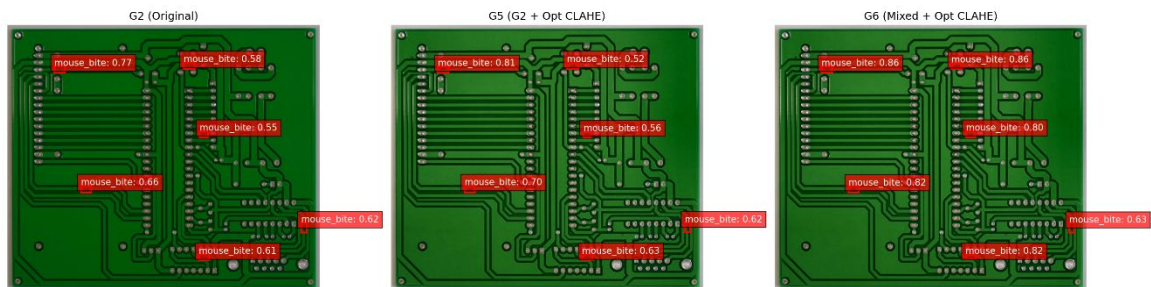


Figure 2.9 Detection results of G2, G5, and G6 for mouse bite defects in
05_mouse_bite_07

2.2.6.3 Short Defect Case: G2 vs G6 Comparison

Figure 2.10 shows another short defect case, named `detection_comparison_short`. This image contains five short defects. G2 detected four of them, while G6 detected all five defects. For the defects detected by both models, the predicted boxes were located around the correct defect regions. The confidence scores of these common detections are compared in Table 2.10.

Defect ID	G2 Confidence	G6 Confidence	Change
1	0.75	0.78	+0.03
2	0.87	0.86	-0.01
3	0.80	0.78	-0.02
4	0.54	0.55	+0.01
5	Not detected	0.66	-

Table 2.10 Confidence scores of G2 and G6 for common short defects in `detection_comparison_short`

For the four defects detected by both models, the confidence scores of G2 and G6 were generally close. For Defect 4, G2 gave a confidence score of 0.54, while G6 gave a slightly higher score of 0.55. Although the increase was small, this defect was in the relatively low-confidence range for both models. More importantly, G6 also detected one additional short defect that was not detected by G2, with a confidence score of 0.66. This case suggests that the G6 strategy may help improve recall in some short defect images. Since G6 was trained with both original images and optimal CLAHE-enhanced images, the model may have been better adapted to the enhanced image style used during inference. As a result, it was able to identify one defect that was missed by the original augmented model in this example.

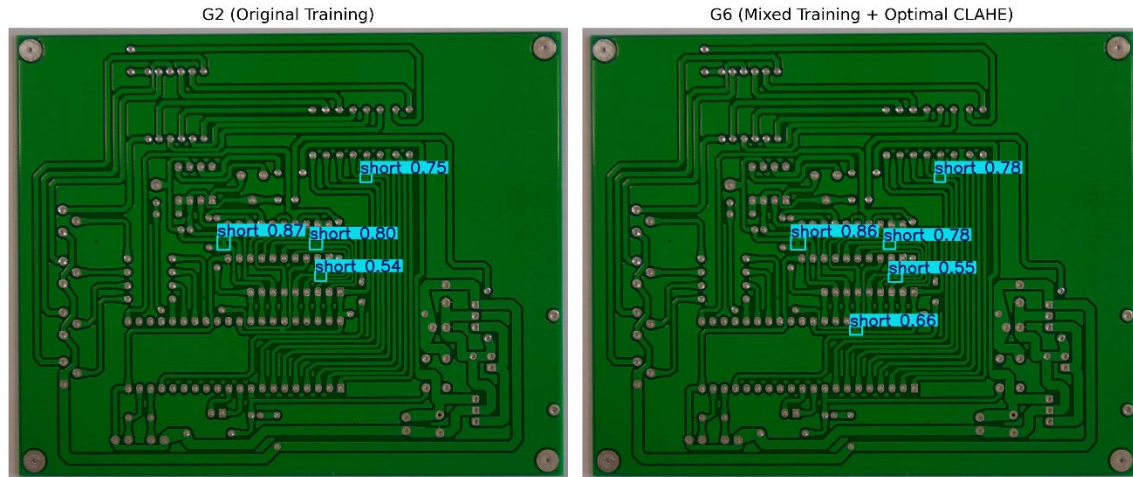


Figure 2.10 Detection results of G2 and G6 for short defects in
detection_comparison_short

2.2.6.4 Mouse Bite Defect Case: G2 vs G6 Comparison

Figure 2.11 shows the detection results for the image detection_comparison_mb. This image contains five mouse bite defects. G2 detected two of them, while G6 detected three. The detected defects were located around the correct defect regions. Table 2.11 lists the confidence scores for the two defects detected by both models.

Defect ID	G2 Confidence	G6 Confidence	Change
1	0.55	0.83	+0.28
2	0.82	0.80	-0.02
3	Not detected	0.79	-

Table 2.11 Confidence scores of G2 and G6 for common mouse bite defects in
detection_comparison_mb

For Defect 1, G2 gave a confidence score of 0.55, which was relatively low. In comparison, G6 increased the confidence score to 0.83, corresponding to an improvement of 50.9%. In addition, G6 detected one more mouse bite defect that was not detected by G2, with a confidence score of 0.79. This example suggests that the training-inference preprocessing

consistency strategy may be helpful for mouse bite defects, especially when the defect features are weak and the original model gives low-confidence predictions. Since G6 was trained with both original images and optimal CLAHE-enhanced images, it was more familiar with the enhanced image style used during inference. This may explain why it produced a higher confidence score for the common defect and detected one additional target in this case.

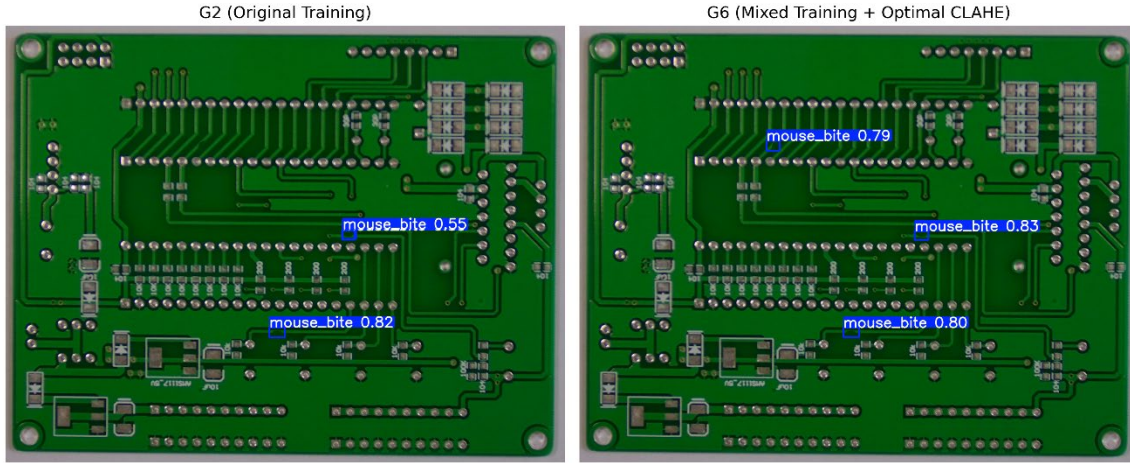


Figure 2.11 Detection results of G2 and G6 for mouse bite defects in
detection_comparison_mb

2.2.6.5 Summary

The four visual cases provide additional examples to support the quantitative results discussed in Sections 2.2.1 to 2.2.5. Overall, G2 showed good detection ability as the augmented baseline model and was able to detect most clearly visible defects. G5 sometimes improved the confidence scores after using the optimal CLAHE parameters during inference, but its improvement was not consistent across all cases. This agrees with the earlier result that inference-stage parameter optimization can reduce part of the performance loss, but it cannot fully solve the mismatch between training and inference image styles.

Compared with G2 and G5, G6 generally produced more stable confidence scores in the selected examples. In some cases, it also detected additional defects that were missed by G2, especially for small and low-contrast mouse bite defects. These visual results suggest that adding optimal CLAHE-enhanced images into the training set helped the model adapt better to the preprocessing style used during inference. Taken together, these image examples give the same message as the numerical results: when the model is trained with the same CLAHE setting used during inference, its detection results become more reliable, especially for small defects with weak contrast or complex surrounding backgrounds.

2.2.7 Summary

In this section, the six experimental groups from G1 to G6 were compared. The purpose was to check how the detection results changed when data augmentation, CLAHE preprocessing, CLAHE parameter optimization, and mixed training were used. The main results can be summarized as follows.

(1) Data augmentation improved the baseline result.

G1 was trained without data augmentation, while G2 used the default augmentation setting in YOLOv8. On the test set, the mAP50 increased from 0.797 in G1 to 0.907 in G2, giving a relative improvement of 13.8%. This result shows that the default data augmentation settings in YOLOv8 were useful for improving the detection of PCB defects in this dataset, especially under the condition of limited training images.

(2) Applying default CLAHE only during inference reduced detection performance.

When default CLAHE parameters were used during inference, both G3 and G4 showed lower mAP50 values than their corresponding original-image groups. Specifically, G3 decreased by 9.3% compared with G1, and G4 decreased by 7.4% compared with G2. The effect was more obvious for mouse bite defects, whose AP decreased by 11.4% to 11.7%. These results suggest that changing the image style only at the inference stage

caused a mismatch between the training and test images. This observation is consistent with the general explanation of domain shift, where a difference between training and testing distributions may lead to performance loss^[24, 38].

(3) CLAHE parameter optimization reduced part of the performance loss, but its effect was limited.

Through grid search, the optimal CLAHE parameters in this study were identified as clipLimit = 1.0 and tileGridSize = (6,6). Compared with G4, which used the default CLAHE parameters, G5 improved mAP50 by 6.5%. However, G5 still remained slightly lower than G2, with mAP50 values of 0.895 and 0.907, respectively. The parameter analysis in this study showed that lower contrast limit values, especially clipLimit $\leq$ 2.0, were more suitable for the current PCB defect images. This is reasonable because CLAHE studies also show that the contrast limit can affect the balance between contrast improvement and noise control^[31]. In contrast, higher clipLimit values tended to over-enhance background texture and noise, which could interfere with defect detection^[13]. Therefore, parameter optimization can make the CLAHE-enhanced images more suitable for the fixed model, but it cannot fully remove the distribution difference between training and inference images^[24].

(4) The training-inference preprocessing consistency strategy achieved the best overall result.

G6 used a mixed training set containing both original images and optimal CLAHE-enhanced images. During inference, the same optimal CLAHE parameters were applied to the test images. With this design, G6 achieved the highest mAP50 on the test set, reaching 0.910. This was 1.7% higher than G5 and 0.3% higher than G2. The result suggests that including the target preprocessing style during training helped the model adapt better to CLAHE-enhanced images. At the same time, the expanded training set also increased data diversity, which may have contributed to the improvement^[21, 38].

(5) The visual cases were consistent with the quantitative results.

The four visual examples further showed the differences among G2, G5, and G6. G2 performed well as the augmented baseline model. G5 improved some confidence scores after using the optimal CLAHE parameters, but the improvement was not consistent across all defects. In comparison, G6 generally gave higher or more stable confidence scores in the selected cases and detected additional defects in some images. This was especially clear for small and low-contrast mouse bite defects. The visual examples therefore gave the same message as the numerical results: using the optimal CLAHE parameters only during inference was useful, but including the same preprocessing style during training gave a more stable result.

In this section, the results showed that preprocessing choices changed the detection performance of YOLOv8n on the PCB defect dataset. Data augmentation raised the baseline result, while default CLAHE used only during inference reduced the performance. After the CLAHE parameters were optimized, part of this loss was recovered. However, the best result came from G6, where images processed with the optimal CLAHE parameters were also added to the training set. For a PCB defect detection system with a fixed preprocessing step, this means that the preprocessing used at deployment should be considered during training, not only added after the model has already been trained.

Chapter 3

Conclusion and Future Work

3.1 Conclusion

3.1.1 Main Experimental Findings

In this project, six experimental groups, from G1 to G6, were used to test how different preprocessing choices affected YOLOv8n-based PCB defect detection. The experiments focused on two defect types: mouse bite defects and short defects. From the test-set results, the first clear change was found between G1 and G2. G1 was trained without data augmentation, while G2 used YOLOv8 default augmentation. After augmentation was enabled, the mAP50 increased from 0.797 to 0.907, giving a relative improvement of 13.8%. This shows that, for the current dataset, the default augmentation setting helped the model detect PCB micro-defects more effectively.

The results also showed that adding CLAHE only during inference did not always improve the detection result. When default CLAHE parameters were used, G3 and G4 showed mAP50 decreases of 9.3% and 7.4% compared with G1 and G2, respectively. The decrease was more obvious for mouse bite defects, whose AP values dropped by 11.4% to 11.7%. This suggests that changing the image style during inference, while keeping the training images unchanged, can lead to a mismatch between training and testing data. This observation is consistent with the general explanation of domain shift^[24, 38].

To reduce this loss, CLAHE parameter optimization was carried out through grid search. In this study, the optimal CLAHE parameters were identified as clipLimit = 1.0 and tileGridSize = (6,6)^[31]. Compared with G4 using default CLAHE parameters, G5 improved mAP50 by 6.5%. However, its result was still 1.2% lower than G2. This means that

inference-stage parameter adjustment can make CLAHE-enhanced images more suitable for the fixed model, but it cannot fully solve the distribution difference between training and inference images when the model itself is still trained only on original images^[24].

3.1.2 Effect of Training-Inference Preprocessing Consistency

The best overall performance was obtained by G6. Different from G5, G6 did not only adjust CLAHE parameters during inference. Instead, it used a mixed training set containing both original training images and optimal CLAHE-enhanced images. During inference, the same optimal CLAHE parameters were applied to the test images. With this design, G6 achieved the highest mAP50 of 0.910, which was 1.7% higher than G5 and 0.3% higher than G2.

This result shows that training-inference preprocessing consistency is important when a fixed preprocessing method is used at the inference stage. Since G6 had already learned from optimal CLAHE-enhanced images during training, it was better adapted to the image style used during testing. At the same time, the mixed training set also increased the diversity of training images, which may have contributed to the performance improvement^[38].

The class-level results give a more detailed view of the difference between G2, G5, and G6. For mouse bite defects, G6 reached an AP of 0.877, higher than both G2 and G5. This improvement is important because earlier comparisons showed that mouse bite defects were more sensitive to preprocessing changes. The result for short defects was different. In G6, the AP for short defects was 0.942, lower than 0.959 in G2 and slightly lower than 0.949 in G5. This means that CLAHE-based preprocessing did not bring the same benefit to every defect category. Short defects may already be relatively clear in the original images, so additional contrast enhancement did not improve their AP. Even with this

limitation, G6 still achieved the highest overall mAP50 and the best AP for mouse bite defects. Therefore, G6 offered the best overall trade-off among the six experimental groups.

The visual examples also support this result pattern. G5 improved the confidence scores of some defects after using the optimal CLAHE parameters, but the improvement was not stable across all selected cases. G6 generally produced more stable confidence scores and detected additional targets in some images, especially in the mouse bite examples. These observations show that using optimal CLAHE parameters only during inference was helpful but still limited. When the same CLAHE processing style was also included during training, the model produced more stable predictions. This supports the main finding of this study: the optimal CLAHE parameters identified in G5 were useful, and their effect became stronger when they were combined with training-inference preprocessing consistency in G6.

3.1.3 Practical Implications for PCB Defect Detection

The experiments in this project show that preprocessing should be treated as part of the detection pipeline, not just as a simple preparation step. For PCB images with small defects, unsuitable preprocessing settings can lower detection accuracy even when the model itself is unchanged. CLAHE may help make local defect details clearer, but if the parameters are too strong, or if CLAHE is used only during inference, background texture and circuit patterns may also become more obvious. In that case, the detector may have more difficulty separating defects from the PCB background.

For this reason, the CLAHE parameter search in G5 is an important part of this project. Instead of directly using the default CLAHE setting, different combinations of clipLimit and tileGridSize were tested. The best setting found in this study was clipLimit = 1.0 and tileGridSize = (6,6). This result shows that mild contrast enhancement was more suitable for the current PCB defect dataset than stronger enhancement. For practical inspection

systems, this means that CLAHE parameters should be tested on the target dataset before deployment, rather than simply copied from default settings.

The results also show that finding suitable parameters is not enough by itself. In G5, the optimized CLAHE parameters reduced part of the performance loss, but the model had still been trained only on original images. In G6, original images and optimal CLAHE-enhanced images were both included in the training set, and the same CLAHE setting was used during inference. This made the training and inference image styles more consistent, and the model achieved the best overall mAP50. Therefore, if a fixed preprocessing pipeline is planned for deployment, similar preprocessing should also be included during model training.

Moreover, the interaction between data augmentation and CLAHE also matters. Data augmentation helps the model learn from more image variations, while CLAHE changes the contrast and local texture of the input images. In G6, these two factors worked together: YOLOv8 default augmentation was still used, and the training set was expanded with images processed by the optimal CLAHE parameters. This gave the model the opportunity to learn both original and enhanced image styles. For PCB defect detection tasks with small targets, low contrast, and fixed preprocessing parameters, this design provides a practical baseline for setting up the training pipeline.

3.2 Future Work

This project tested several preprocessing settings for PCB defect detection, but the experiments were limited by dataset size, defect category coverage, and the use of a fixed YOLOv8n model. The work can be extended in the following directions.

3.2.1 Dataset Expansion and Augmentation Strategy Exploration

The first limitation is the dataset. This project only used two defect categories, mouse bite defects and short defects, with 231 images in total. These two categories were selected because they are small and relatively difficult to detect, but they do not cover all common PCB defects. Future work should include all six defect categories, such as missing hole, mouse bite, open circuit, short, spur, and spurious copper. More images should also be collected from different PCB boards, production lines, imaging devices, lighting conditions, and surface backgrounds. A larger and more varied dataset would allow the training-inference preprocessing consistency strategy to be tested under conditions closer to real industrial inspection.

The source of the performance improvement in G6 also needs further separation. In this project, G6 used a mixed training set containing original images and images processed with the optimal CLAHE parameters. As a result, the training set increased in size, and the visual style of the samples also became more diverse. The current experiment does not fully distinguish between the effect of increased data volume and the effect of increased visual diversity. Future work should design more detailed ablation experiments to separate these two factors. For example, one control group could duplicate the original images to increase data volume without changing image style. Another group could use transformed or enhanced images while keeping the total number of training samples comparable. A third group could combine both larger data volume and more visual diversity. Comparing these groups on a larger dataset would make the contribution of data volume and data diversity clearer.

Future work should also evaluate different augmentation and enhancement strategies more systematically. Besides common operations such as flipping, scaling, cropping, and color adjustment, CLAHE-based enhancement can be tested as a deterministic image-level transformation. Generative methods, such as GANs or diffusion models, may also be explored because they may generate more realistic defect variations when real defect

samples are limited. These comparisons would help identify which type of augmentation is more suitable for small and low-contrast PCB defects.

3.2.2 Model Improvement and Real-Time Deployment

This project used YOLOv8n as the baseline model, so that different preprocessing strategies could be compared under the same detection framework. YOLOv8n-based models have also been used in recent PCB micro-defect detection studies^[21]. However, the current conclusion is still based on one lightweight detector. Future work should test the same training-inference preprocessing consistency strategy on other architectures, such as YOLOv9, YOLOv10, other YOLOv8 variants, or Transformer-based detectors. Results from different model structures would strengthen the generalizability of the conclusion.

The CLAHE parameters in this project were determined through grid search. The optimal setting, clipLimit = 1.0 and tileGridSize = (6,6), worked best for the current dataset, but it is not guaranteed to be optimal across all PCB images. Some images may contain stronger background texture, lower brightness, or weaker defect boundaries. One direction is to design adaptive preprocessing. For example, a lightweight module could select CLAHE parameters according to the contrast, brightness, or texture level of each input image, instead of applying one fixed setting to all images.

Real-time deployment also requires further evaluation. Due to the lack of a compatible GPU environment, all experiments in this project were run on CPU. This setting was sufficient for relative comparison among G1 to G6, but it did not evaluate the real-time speed of the method. Future work should use GPU acceleration and report detection speed, such as FPS, under realistic production-line conditions. These methods could enable deployment of both the detection model and the preprocessing pipeline on edge

devices such as the NVIDIA Jetson series for online PCB inspection, as edge deployment has also been tested in PCB defect detection studies^[22].

3.2.3 Generalization to Other Domains and Industrial Vision Tasks

This project mainly studied a simple case of domain shift: the image style change caused by using CLAHE only during inference. This problem is related to the broader domain shift problem discussed in test-time adaptation studies^[24]. In real manufacturing environments, domain shift can also come from production line differences, material batches, camera sensors, illumination settings, and equipment aging. Future work should combine the preprocessing consistency strategy with domain adaptation or domain generalization methods^[38]. This would test whether the model can maintain performance when the inspection environment changes over time.

The same idea can also be examined in other industrial vision tasks. Surface scratch detection, weld seam inspection, and semiconductor wafer defect identification all involve small targets, weak contrast, or complex background texture. These tasks would serve as suitable test cases for checking whether training-inference preprocessing consistency remains effective outside PCB defect detection. If similar results are obtained, the strategy could serve as a practical reference for designing preprocessing pipelines in industrial machine vision.

Future work should validate these findings under broader and more realistic conditions, including larger datasets, clearer ablation designs, adaptive preprocessing, GPU-based speed evaluation, and edge-device deployment. These steps are needed before the proposed preprocessing strategy can be translated into practical automated optical inspection systems.

References

- [1] Q. Wang, M. Wang, J. Sun, D. Chen, and P. Shi, “Review of Surface-Defect Detection Methods for Industrial Products Based on Machine Vision,” *IEEE Access*, vol. 13, pp. 90668–90697, 2025.
- [2] Y. Bian, P. Du, and L. Song, “A PCB Defect Detection Algorithm YOLOv8 Combined Adaptive Feature Fusion Scheme and Neck-Head Bridge,” in *Proceedings of the 44th Chinese Control Conference*, Chongqing, China, 2025, pp. 7467–7473.
- [3] J. Wang, D. Wang, L. Feng, and C. Shen, “PCB Small Object Defect Detection Model Based on Improved YOLOv8s,” *Comput. Eng. Appl.*, vol. 61, no. 15, pp. 288–297, 2025.
- [4] Z. Yan, R. Hao, B. Huang, L. Zhu, and H. Pan, “A Domain Incremental Learning Framework for PCB Continuous Defect Detection,” *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–13, 2025.
- [5] Z. He, Y. Lian, Y. Wang, and Z. Lu, “A comprehensive review of research on surface defect detection of PCBs based on machine vision,” *Results Eng.*, vol. 27, 2025.
- [6] D. Wan and M. B. Abisado, “Hybrid Convolutional Neural Network with Transfer Learning and MobileNetV2 for Advanced Multi-Class PCB Defect Detection,” in *2025 8th International Conference on Information and Computer Technologies (ICICT)*, 2025, pp. 140–143.
- [7] J. Li, S. Wang, T. Lei, Y. Yu, Y. Liu, and Y. Liu, “An Improved Feature Extraction and Fusion Model for PCB Surface Defect Detection,” in *2025 6th International*

Conference on Internet of Things, Artificial Intelligence and Mechanical Automation (IoTAIMA), 2025, pp. 68–73.

- [8] A. Jain, K. Patel, A. Topiwala, R. K. Hegde, and S. Pandya, “Automated Defect Detection in Unmasked Printed Circuit Boards using YOLOv8,” in *2025 IEEE Space, Aerospace and Defence Conference (SPACE)*, 2025, pp. 1–7.
- [9] T. Wang, L. Wang, T. Wang, Z. Sha, and H. Jiang, “Lightweight PCB Defect Detection Algorithm Based on Improved YOLOv8s,” *J. Electron. Meas. Instrum.*, vol. 39, no. 3, pp. 46–54, 2025.
- [10] R. Ammar, N. Aboshoshsa, S. Aboshoshsa, F. E. Abd El-Samie, A. Abou Elazm, G. M. El-Banby, W. El-Shafai, A. A. Khalaf, and A. El-Safrawy, “Hybrid Method for Contrast Enhancement of Industrial Videoscope Images,” in *2nd IEEE International Conference on Electronic Engineering (ICEEM)*, Menoufia, Egypt, 2021, pp. 1–4.
- [11] X. Liao, J. Liao, and S. Lyu, “Defect Detection for PCB by Combining Shallow Features and Attention Mechanism,” *Comput. Integr. Manuf. Syst.*, vol. 30, no. 3, pp. 1092–1104, 2024.
- [12] R. Ammar, A. Ammar, H. Ashiba, W. El-Shafai, A. Abou Elazm, and F. Abd El-Samie, “Contrast Enhancement of Power Plant Images Obtained from Industrial Borescope Devices,” *Menoufia J. Electron. Eng. Res.*, vol. 29, no. 1, pp. 91–97, 2020.
- [13] V. Gaidhane and S. Kolhe, “Enhanced Weld Defect Detection Through Advanced Image Preprocessing Techniques,” in *2025 International Conference on Electronics, Computing, Communication and Control Technology (ICECCC)*, 2025, pp. 1–6.

- [14] S.-H. Park and S.-H. Lee, “Reference Image-Based Contrastive Attention Mechanism for Printed Circuit Board Defect Classification,” *J. KIISE*, vol. 52, no. 1, pp. 70–76, 2025.
- [15] U. Nisa, M. S. M. Pozi, and M. A. Saip, “A systematic image augmentation framework based on dataset limitations for small object detection,” *Iran J. Comput. Sci.*, vol. 9, no. 1, 2026.
- [16] Q. Pan, T. Liu, Y. Hou, J. Chang, and H. Yang, “An adversarial network based on anomaly domain decomposition and transformation for industrial PCBA defect inspection,” *Neural Comput. Appl.*, vol. 37, no. 18, pp. 12653–12671, 2025.
- [17] G. Sun, Z. Xiong, and Y. Yuan, “Detail-Preserving and Diverse Image Translation for Adverse Visual Object Detection,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 10, pp. 9139–9152, Oct. 2024.
- [18] Z. Wang, Y. Luo, L. Zheng, Z. Chen, S. Wang, and Z. Huang, “In Search of Lost Online Test-Time Adaptation: A Survey,” *Int. J. Comput. Vis.*, vol. 133, no. 3, pp. 1106–1139, 2024.
- [19] S. Yang, L. Yang, Y. Yang, and C. Guo, “Review of PCB Defect Detection Algorithm Based on Machine Vision,” *J. Front. Comput. Sci. Technol.*, vol. 19, no. 4, pp. 901–915, 2025.
- [20] S. Nasri, N. Ahmad, Q. U. A. Aini, A. Qayyum, and N. Ul Islam, “A YOLOv9: Deep Learning-based Framework Defect Detection Method for PCBs,” *J. Electron. Test.*, vol. 41, no. 4, pp. 545–559, 2025.

- [21] X. Song and X. Song, “SNF-YOLOv8: A Lightweight PCB Defect Detection Algorithm base on Multiscale Feature Fusion and Attention Scale Sequence Fusion,” *J. Electron. Test.*, vol. 41, no. 4, pp. 561–573, Aug. 2025.
- [22] V. Kumar Ancha, F. N. Sibai, V. Gonuguntla, and R. Vaddi, “Utilizing YOLO Models for Real-World Scenarios: Assessing Novel Mixed Defect Detection Dataset in PCBs,” *IEEE Access*, vol. 12, pp. 100983–100990, 2024.
- [23] C. Kanellakis, P. Kanellakis, and G. Nikolakopoulos, “On Image Based Enhancement for 3D Dense Reconstruction of Low Light Aerial Visual Inspected Environments,” in *Science and Information Conference*, 2019, pp. 265–279.
- [24] Y. Tang, S. Chen, Z. Kan, Y. Zhang, Q. Guo, and Z. He, “Learning Visual Conditioning Tokens to Correct Domain Shift for Fully Test-time Adaptation,” *IEEE Trans. Multimedia*, pp. 1–12, 2024.
- [25] Y. Tang, S. Tang, Z. Lu, X. Wang, and Z. He, “Dual-Path Adversarial Lifting for Domain Shift Correction in Online Test-Time Adaptation,” in *European Conference on Computer Vision (ECCV)*, 2024, pp. 342–359.
- [26] B. Zhou, Y. Shi, T. Xu, and H. Luo, “Research on Defect Detection Method for PCB Components Based on Improved YOLOv8,” in *2024 3rd International Conference on Advanced Sensing, Intelligent Manufacturing (ASIM)*, 2024, pp. 83–89.
- [27] W. Huang and P. Wei, “A PCB Dataset for Defects Detection and Classification,” *J. Latex Class Files*, vol. 14, no. 8, pp. 1–9, Aug. 2018.

- [28] M. Calabrese, L. Agnusdei, G. Fontana, G. Papadia, and A. Del Prete, “Application of Mask R-CNN and YOLOv8 algorithms for defect detection in printed circuit board manufacturing,” *Discov. Appl. Sci.*, vol. 7, no. 4, 2025.
- [29] H. D. Nguyen, D. Cheng, X. Wang, Y. Shi, and B. Wen, “Data-Efficient Deep Learning for Printed Circuit Board Defect Detection Using X-Ray Images,” *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–14, 2025.
- [30] C. O’Donovan, I. Popov, G. Todeschini, and C. Giannetti, “Ladle pouring process parameter and quality estimation using Mask R-CNN and contrast-limited adaptive histogram equalisation,” *Int. J. Adv. Manuf. Technol.*, vol. 126, no. 3-4, pp. 1397–1416, 2023.
- [31] D. Zhang, A. Alkhayyat, G. Sudhamsu, P. K. Sahu, S. Dasi, A. Srivastava, D. Singh, and J. A. Mayan, “CLAHE-Based Contrast Improvement and Noise Reduction for Foggy Images,” *Natl. Acad. Sci. Lett.*, 2025.
- [32] I. Majid Mohammed and N. Ashidi Mat Isa, “Contrast Limited Adaptive Local Histogram Equalization Method for Poor Contrast Image Enhancement,” *IEEE Access*, vol. 13, pp. 62600–62632, 2025.
- [33] R. Sharma and A. Kamra, “A Review on CLAHE Based Enhancement Techniques,” in *2023 6th International Conference on Contemporary Computing and Informatics (IC3I)*, 2023, pp. 321–325.
- [34] I. Lashkov, R. Yuan, and G. Zhang, “Edge-Computing-Facilitated Nighttime Vehicle Detection Investigations With CLAHE-Enhanced Images,” *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 13370–13383, Nov. 2023.

- [35] V. L. Narla, G. Suresh, C. S. Rao, M. A. Awadh, and N. Hasan, “A multimodal approach with firefly based CLAHE and multiscale fusion for enhancing underwater images,” *Sci. Rep.*, vol. 14, no. 1, p. 27588, Nov. 2024.
- [36] C. Pramerdorfer and M. Kampel, “A Dataset for Computer-Vision-Based PCB Analysis,” in *14th IAPR International Conference on Machine Vision Applications (MVA)*, Tokyo, Japan, 2015, pp. 378–381.
- [37] Z. Zhang, Y. Zhang, M. Iwata, and S. Yoshida, “Data generation using Pix2Pix to improve YOLO v8 performance in UAV-based Yuzu detection,” *Smart Agric. Technol.*, 2025.
- [38] H. Gao, K. Gao, W. Xie, N. Ye, X. Wang, and Z. He, “Consistency Regularization for Domain Generalization with Logit Attribution Matching,” in *Uncertainty in Artificial Intelligence (UAI)*, 2024.

Appendix A. Core Code

A.1 Training Control

Training for the experiments was performed either via ultra-lyric's command-line or dedicated scripts.

Group	Training Method	Key File/ Command
G1	Command-line	yolo train data=dataset_abs.yaml model=yolov8n.pt epochs=80 batch=16 imgsz=640 augment=no_aug.yaml
G2	Command-line	yolo train data=dataset_abs.yaml model=yolov8n.pt epochs=80 batch=16 imgsz=640
G3	(Same as G1)	CLAHE applied during inference
G4	(Same as G2)	CLAHE applied during inference
G5	(Same as G2)	Optimal CLAHE parameters applied during inference
G6	Dedicated script	python train_g6.py

Table A.1 The control methods for each experimental group

Notes:

1. Dataset configuration file dataset_abs.yaml is given in A.5.2.
2. Augmentation control file no_aug.yaml is given in A.5.1.
3. The G6 training script train_g6.py loads the mixed dataset configuration (dataset_g6.yaml, see A.5.3) and calls YOLO training. Its key code (extracted from train_g6.py) is:

```
# train_g6.py (core part)
from ultralytics import YOLO
```

```

model = YOLO("yolov8n.pt")
model.train(
    data="dataset_g6.yaml", # mixed dataset configuration
    epochs=80,
    batch=16,
    imgsz=640,
    augment=True,
    cos_lr=True,
    patience=20,
    project="runs/G6",
    name="exp"
)

```

The generation of mixed training images is performed by `prepare_clahe_train.py`, whose core functionality is described in A.2.

A.2 CLAHE Preprocessing and Dataset Preparation

A.2.1 CLAHE Preprocessing Function

File location: `/root/pcb_defect_exchange/evaluate_all.py`

This file contains the CLAHE image enhancement function. The actual function name may differ from the assumed one. Based on the file content, the core CLAHE enhancement logic is as follows (use the actual function name from the source file):

```

# The actual function name may be enhance_with_clahe or apply_clahe
def enhance_with_clahe(image, clip_limit=2.0, tile_grid_size=(8,8)):
    lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)

```

```
l_enhanced = clahe.apply(l)
lab_enhanced = cv2.merge((l_enhanced, a, b))
return cv2.cvtColor(lab_enhanced, cv2.COLOR_LAB2BGR)
```

A.2.2 Preparation of G6 Mixed Training Set

File location: /root/pcb_defect_exchange/prepare_clahe_train.py

This script reads the original training images, applies the optimal CLAHE parameters (clipLimit=1.0, tileGridSize=(6,6)), and saves the enhanced images to data/images/train_clahe_best. Core code snippet:

```
# prepare_clahe_train.py (core part)
import cv2
from pathlib import Path

src_dir = Path("data/images/train")
dst_dir = Path("data/images/train_clahe_best")
dst_dir.mkdir(parents=True, exist_ok=True)

for img_path in src_dir.glob("*.jpg"):
    img = cv2.imread(str(img_path))
    img_enhanced = enhance_with_clahe(img, clip_limit=1.0, tile_grid_size=(6,6))
    cv2.imwrite(str(dst_dir / img_path.name), img_enhanced)
```

A.3 CLAHE Parameter Grid Search Script

File location: /root/pcb_defect_exchange/grid_search_clahe.py

This script iterates over parameter combinations and evaluates mAP50 on the validation set using the G2 model. The core logic is as follows (refer to the source file for full implementation):

```

import itertools
from ultralytics import YOLO

clip_limits = [1.0, 1.5, 2.0, 2.5, 3.0, 4.0, 5.0]
tile_grids = [(4,4), (6,6), (8,8), (10,10), (12,12), (16,16)]

model = YOLO("runs/G2/exp/weights/best.pt")

for clip, tile in itertools.product(clip_limits, tile_grids):
    # Apply CLAHE to validation images
    # Call model.val() to compute mAP50
    # Record results
    pass

```

A.4 Visualization Script for Detection Comparisons

File location: /root/pcb_defect_exchange/visualize_comparison_fixed.py

This script generates side-by-side detection results for models G2, G5, and G6 on the same images. The core plotting function (use the actual function name from the source file) is:

```

def plot_detections(image, model, model_name, clahe_params=None):
    img = image.copy()
    if clahe_params:
        img = enhance_with_clahe(img, **clahe_params) # call enhancement function
    from A.2
    results = model(img)[0]
    # Draw bounding boxes and confidence using matplotlib
    # ...
    return fig

```

A.5 Configuration Files

A.5.1 Disable Augmentation (no_aug.yaml)

File location: /root/pcb_defect_exchange/no_aug.yaml

```
hsv_h: 0.0
hsv_s: 0.0
hsv_v: 0.0
degrees: 0.0
translate: 0.0
scale: 0.0
shear: 0.0
perspective: 0.0
flipud: 0.0
fliplr: 0.0
mosaic: 0.0
mixup: 0.0
copy_paste: 0.0
```

A.5.2 Dataset Configuration for Original Images (dataset_abs.yaml)

File location: /root/pcb_defect_exchange/dataset_abs.yaml

```
path: /root/pcb_defect_exchange/data
train: images/train
val: images/val
test: images/test
nc: 2
names: ['mouse_bite', 'short']
```

A.5.3 Dataset Configuration for G6 Mixed Training (dataset_g6.yaml)

File location: /root/pcb_defect_exchange/dataset_g6.yaml

```
# Adjust the val field according to the actual validation set path
path: /root/pcb_defect_exchange/data
train:
  - images/train
  - images/train_clahe_best
val: images/temp_clahe/clip1.0_tile6 # or images/val_clahe_best
names:
  0: mouse_bite
  1: short
```

Appendix B. Supplementary Experimental Results

B.1 Full Grid Search Results

Table B.1 shows the validation results obtained from the CLAHE parameter grid search. Each row corresponds to one combination of clipLimit and tileGridSize. The reported metrics include mAP50, Precision (P), Recall (R), and mAP50-95. Here, mAP50 refers to the average precision when the IoU threshold is 0.5, while mAP50-95 is calculated over IoU thresholds from 0.5 to 0.95. Among all tested settings, clipLimit = 1.0 and tileGridSize = (6,6) produced the best result and are therefore marked in bold.

clipLimit	tileGridSize	mAP50	P	R	mAP50-95
1	(4, 4)	0.834753	0.957121	0.768309	0.834753
1	(6, 6)	0.837436	0.959939	0.769172	0.837436
1	(8, 8)	0.834441	0.970381	0.762782	0.834441
1	(10, 10)	0.832024	0.964755	0.751848	0.832024
1	(12, 12)	0.830044	0.955479	0.767843	0.830044
1	(16, 16)	0.834788	0.952848	0.757741	0.834788
1.5	(4, 4)	0.811237	0.944674	0.739082	0.811237
1.5	(6, 6)	0.808618	0.891134	0.740882	0.808618
1.5	(8, 8)	0.80113	0.901048	0.73856	0.80113
1.5	(10, 10)	0.80233	0.899652	0.735948	0.80233
1.5	(12, 12)	0.803631	0.909587	0.732345	0.803631
1.5	(16, 16)	0.803462	0.942878	0.739583	0.803462
2	(4, 4)	0.777859	0.867784	0.720274	0.777859
2	(6, 6)	0.768805	0.803843	0.732719	0.768805
2	(8, 8)	0.76522	0.813468	0.731034	0.76522
2	(10, 10)	0.768087	0.836325	0.708558	0.768087

2	(12, 12)	0.761079	0.843263	0.711002	0.761079
2	(16, 16)	0.788122	0.884676	0.731034	0.788122
2.5	(4, 4)	0.752185	0.882984	0.677492	0.752185
2.5	(6, 6)	0.711645	0.765662	0.722699	0.711645
2.5	(8, 8)	0.730675	0.852342	0.685361	0.730675
2.5	(10, 10)	0.721236	0.742812	0.697861	0.721236
2.5	(12, 12)	0.730454	0.841653	0.68546	0.730454
2.5	(16, 16)	0.746651	0.871549	0.646783	0.746651
3	(4, 4)	0.723531	0.841597	0.668157	0.723531
3	(6, 6)	0.705458	0.837252	0.665146	0.705458
3	(8, 8)	0.702204	0.834698	0.659442	0.702204
3	(10, 10)	0.70324	0.778714	0.679475	0.70324
3	(12, 12)	0.695169	0.798425	0.666585	0.695169
3	(16, 16)	0.714981	0.811274	0.645156	0.714981
4	(4, 4)	0.653271	0.740047	0.655246	0.653271
4	(6, 6)	0.659418	0.697372	0.654367	0.659418
4	(8, 8)	0.647331	0.723049	0.656701	0.647331
4	(10, 10)	0.646973	0.727474	0.640409	0.646973
4	(12, 12)	0.642062	0.675531	0.652299	0.642062
4	(16, 16)	0.637798	0.680024	0.63087	0.637798
5	(4, 4)	0.632854	0.736838	0.625123	0.632854
5	(6, 6)	0.607428	0.692677	0.597823	0.607428
5	(8, 8)	0.596424	0.678701	0.601425	0.596424
5	(10, 10)	0.593746	0.613771	0.625111	0.593746
5	(12, 12)	0.585744	0.588442	0.599343	0.585744
5	(16, 16)	0.571867	0.680703	0.547318	0.571867

Table B.1 Full Grid Search Results

Appendix C. Environment and Configuration

C.1 Software Environment

The experiments were conducted under the following software environment:

Package	Version
Operating System	Ubuntu (Linux kernel based on 5.4)
Python	3.12.11
ultralitics	8.4.16
torch	2.91+cu130
opencv-python	4.8.1
numpy	1.24.3
pandas	2.0.3
matplotlib	3.7.2
tqdm	4.66.1

Table C.1 Statement of Software Environment

C.2 Hardware Specifications

Due to GPU compatibility issues, all experiments were performed on CPU. However, the unified hardware environment ensures comparability across experimental groups.

Component	Specification
CPU	Intel Xeon Gold 5118 @ 2.30GHz (12 cores)
RAM	32 GB DDR4
Storage	1 TB NVMe SSD
GPU	Not used (CPU-only mode)

Table C.2 Statement of Hardware Specification

C.3 Training Hyperparameters

All experimental groups (G1–G6) used identical hyperparameter settings to ensure comparability of results. The details are as follows:

Parameter	Value
Optimizer	SGD
Initial learning rate	0.01
Momentum	0.937
Weight decay	0.0005
Learning rate schedule	Cosine annealing (cos_lr=True)
Batch size	16
Epochs	80
Input image size	640 × 640 pixels
Early stopping patience	20 epochs
Data augmentation (G2/G4/G6)	YOLOv8 default (Mosaic, random flip, scale, crop, color jitter, etc.)
Data augmentation (G1/G3/G5)	Fully disabled (via no_aug.yaml configuration)

Table C.3 Statement of Training Hyperparameters